BEGINNER'S GUIDE:

HOW MACHINES SENSE, CONNECT, AND ACT

BLUE IMPULSE

Your route through the guide

Six chapters, one connected story

Chapter	Read
01 · The Fundamentals	Pages 5–21 · Dawn
02 · Behaviorism & Behaviorist AI	Pages 22–33 · Morning
03 · Hardware	Pages 34–50 · Midday
04 · Networking	Pages 51–70 · Afternoon
05 · Applications	Pages 71–83 · Dusk
06 · Strengths & Limitations	Pages 84–95 · Midnight

How to study

Start with a chapter's learning goals, then read one teaching page at a time. Follow the diagram or worked example. Pause at the question before moving on, and write an answer in your own notes. Then try the matching chapter mission in the activity section beginning on page 97. Explained answers follow the six missions.

Find a page quickly

This PDF has bookmarks for every chapter and teaching page. Page numbers match the PDF viewer, including the unchanged cover and chapter dividers. The learning goals are on pages 3–4. Missions, optional resources and the complete answer key follow page 96; their bookmarks provide direct navigation. GitHub links are supplementary.

PAUSE & TRY

Define a smart system and explain how it integrates physical hardware, network connectivity and software into one workflow.

Chapter 5 · Question 1 · Adapted from GitHub exercises

What you will learn

01 · The Fundamentals

Represent numbers, characters, images and sound; explain stored programs, registers and the fetch–execute cycle; compare translators; describe operating-system tasks, utilities, interrupts and peripheral connections.

02 · Behaviorism & Behaviorist AI

Distinguish classical and operant conditioning; identify stimulus–response rules and layered robotics; explain reinforcement-learning components, policies, values and exploration; interpret a Q update and the parts of an MDP.

03 · Hardware

Classify hardware; compare memory and storage; explain processors, pipelining, parallelism and interrupts; connect sensors, converters and actuators; reason with logic gates; evaluate embedded systems, interfaces and AI accelerators.

PAUSE & TRY

Assemblers, compilers and interpreters are all used to translate or execute programs. Discuss the different roles played by each translator.

Chapter 1 · Question 5 · Adapted from GitHub exercises

From concepts to decisions

04 · Networking

Compare network architectures, clients and clouds; identify communication hardware and media; trace URLs, DNS and TCP/IP; explain application protocols, switching and network performance; distinguish data protection goals, encryption, signatures and TLS.

05 · Applications

Trace an integrated smart-system workflow; explain real-time deadlines and edge/cloud choices; connect policies to actuators; evaluate automation, human-AI interaction, ethical concerns and connected applications.

06 · Strengths & Limitations

Evaluate physical and network constraints; distinguish scalability, interoperability and resilience; compare predictive analytics with behavioural modelling; assess resource allocation, explainability, bias and responsible deployment.

PAUSE & TRY

Explain resource optimisation when balancing local hardware processing against network transmission costs.

Chapter 6 · Question 5 · Adapted from GitHub exercises

DAWN

THE FUNDAMENTALS

1

From bits to bytes

Two states, many meanings

A bit is a binary digit: 0 or 1. Electronic circuits distinguish two ranges of voltage, allowing hardware to store and process bits reliably. Eight bits form one byte. A word is the group of bits a processor handles as a unit; its size depends on the architecture. The same pattern may represent a number, a character or an instruction: the interpretation supplies the meaning.

Measure storage accurately

Prefix	Bytes
kB / MB / GB	1,000 / 1,000,000 / 1,000,000,000
KiB / MiB	1,024 / 1,048,576
GiB / TiB	1,073,741,824 / 1,099,511,627,776

Units matter

Lowercase b means bit; uppercase B means byte. Divide bits by 8 to obtain bytes. Decimal prefixes use powers of ten; binary prefixes use powers of two. For example, 2 KiB = 2 × 1,024 = 2,048 bytes. A network speed in Mb/s is not the same unit as a file size in MB.

PAUSE & TRY

An image is 16,384 pixels wide and 512 pixels high. It is saved as a 256-colour bitmap. Calculate the raw image-data size in gibibytes, excluding headers and compression.

Chapter 1 · Question 2b3 · Adapted from GitHub exercises

Binary, denary & hex

Place value does the work

Denary uses powers of 10. Binary uses powers of 2, so the positions in an eight-bit number are 128, 64, 32, 16, 8, 4, 2 and 1. Add the positions containing 1: 00101101 = 32 + 8 + 4 + 1 = 45. To convert denary to binary, select those powers of two, or repeatedly divide by 2 and read the remainders backwards.

Hexadecimal groups four bits

Binary nibble	Hex	Denary
0000–1001	0–9	0–9
1010 / 1011 / 1100	A / B / C	10 / 11 / 12
1101 / 1110 / 1111	D / E / F	13 / 14 / 15

Why engineers use hexadecimal

Group binary digits in fours from the right: 0010 1101 becomes 2D. Hexadecimal is a compact notation for the same value, useful in memory dumps, colour codes and hardware addresses. Binary addition carries when a column reaches 2: 1 + 1 = 10. With a fixed width, check whether a carry or signed overflow changes the interpretation.

PAUSE & TRY

The byte 01001111 represents an eight-bit two’s complement integer. State its equivalent denary value.

Chapter 1 · Question 1a1 · Adapted from GitHub exercises

Represent negative values

Three eight-bit representations of −5

Method	Representation	Rule
Sign and magnitude	10000101	Sign bit + magnitude
One's complement	11111010	Invert the bits of +5
Two's complement	11111011	Invert +5, then add 1

Read two's complement

The leftmost bit has a negative weight. In eight bits, 11111011 means −128 + 64 + 32 + 16 + 8 + 2 + 1 = −5. In n bits, the range is −2^(n−1) to 2^(n−1) − 1. Always state the width: a bit pattern has no unique signed meaning without a representation and a width.

Subtract by adding the negative

To calculate 9 − 3 in eight bits, add 00001001 to the two's complement of 00000011: 11111101. Keep the eight result bits, giving 00000110. A carry outside the width is discarded. Signed overflow occurs when the true result lies outside the representable range; it is not the same as an unsigned carry.

PAUSE & TRY

Write the integer −53 in eight-bit two's complement form.

Chapter 1 · Question 1a3 · Adapted from GitHub exercises

Numbers, characters & codes

BCD stores one decimal digit at a time

Binary-coded decimal allocates four bits to each decimal digit. Decimal 259 becomes 0010 0101 1001. This is different from converting the whole number 259 to ordinary binary. Patterns 1010–1111 are unused as decimal digits in ordinary BCD. Decimal representations are useful where exact decimal digits matter, including displays and some financial calculations.

Characters need an agreed code

A character set defines the available characters. An encoding maps characters to stored values. ASCII defines 128 codes, covering English letters, digits, punctuation and control characters. Unicode assigns code points to characters from many writing systems. UTF-8 and UTF-16 are ways to encode those code points; Unicode does not mean every character occupies the same number of bytes.

One pattern, different interpretations

Bits → **Encoding / format** → **Meaning**

An application must know the agreed format before it can interpret stored data.

PAUSE & TRY

Write the denary number represented by the BCD sequence 1001 0111 0111 0110.

Chapter 1 · Question 1b2 · Adapted from GitHub exercises

Build a digital image

Pixels and colour depth

A bitmap is a grid of pixels. Pixel dimensions describe its width and height; resolution may also describe pixel density when an image is displayed or printed. Colour depth is the number of bits used per pixel. With b bits per pixel, up to 2^b different codes are available. More pixels preserve finer spatial detail; more bits per pixel allow more colour or intensity values.

Uncompressed bitmap size

Quantity	Example
Dimensions	1,000 × 500 pixels
Colour depth	8 bits per pixel
Raw pixel data	1,000 × 500 × 8 = 4,000,000 bits
In bytes	4,000,000 ÷ 8 = 500,000 bytes

Bitmap versus vector

A vector image stores shapes, paths and their properties. It can scale without pixelation, making it useful for diagrams and logos. Photographs are normally bitmaps. The size formula above excludes headers, palettes, metadata and compression; an actual file may have a different size.

PAUSE & TRY

A developer uses 16-colour bitmap images. State the number of bits required to encode the data for one pixel.

Chapter 1 · Question 2b2 · Adapted from GitHub exercises

Sample, store & compress

Turn an analogue wave into numbers

An analogue-to-digital converter samples a sound signal. Sampling rate is the number of samples per second, in hertz. Sampling resolution, or bit depth, is the number of bits per sample. A higher rate can represent a wider range of frequencies; a greater bit depth provides finer amplitude steps. Both increase uncompressed storage requirements.

Estimate audio size

Uncompressed audio size in bits = sampling rate × bit depth × channels × duration. For 10 seconds of mono audio at 8,000 samples/s and 16 bits/sample: 8,000 × 16 × 1 × 10 = 1,280,000 bits, or 160,000 bytes. Stereo requires two channels.

Choose the right compression

Lossless	Lossy
Reconstructs the original data exactly.	Permanently discards some information.
ZIP, run-length encoding, PNG.	JPEG images, MP3 audio.
Suitable for programs and exact records.	Useful when smaller media files justify reduced fidelity.

PAUSE & TRY

Describe how sampling resolution affects the accuracy of stored digitised sound.

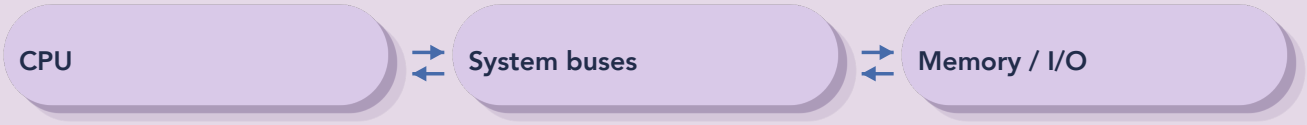
Chapter 1 · Question 2a2 · Adapted from GitHub exercises

The stored-program computer

Instructions and data share memory

In the introductory Von Neumann model, memory holds both instructions and data. The CPU fetches instructions, decodes them and performs the required operations. The arithmetic logic unit (ALU) performs calculations and comparisons. The control unit coordinates the operation of the CPU, memory and input/output devices. Registers hold small amounts of immediately needed information.

The main connections



Address bus: identifies a location. Data bus: carries instructions or values. Control bus: carries signals such as read and write.

Timing and the bottleneck

The system clock coordinates operations. A clock frequency of 3 GHz means three billion clock cycles per second, not necessarily three billion completed instructions. Cache and pipelining reduce some delays, but moving instructions and data through shared resources can still restrict throughput. Real processors extend this simplified model in many ways.

PAUSE & TRY

Put these fetch–execute stages in order: copy MDR to CIR; execute the instruction; decode the instruction; copy PC to MAR; increment PC by 1; copy the instruction at the memory location in MAR into MDR.

Chapter 1 · Question 3 · Adapted from GitHub exercises

Fetch / decode / execute

Follow one instruction

Step	Register transfer or action
1	Copy the address in PC to MAR.
2	Read the memory instruction into MDR.
3	Advance PC to the next instruction address.
4	Copy the instruction from MDR to CIR.
5	Decode the operation and its operand.
6	Execute; store a result or change control flow.

Give each register a job

PC is the program counter: the next instruction address. MAR identifies the memory location being accessed. MDR holds data moving to or from memory. CIR holds the current instruction. ACC is an accumulator for intermediate arithmetic results in this teaching model. A branch instruction may replace the normal next value of PC.

A useful analogy

Think of a recipe: PC marks the next recipe line, MAR selects its location, MDR carries the line, CIR holds the instruction being followed, and ACC is a temporary mixing bowl. In this simplified one-word instruction model PC advances by 1; actual instruction sizes and timing depend on the processor.

PAUSE & TRY

Put these fetch–execute stages in order: copy MDR to CIR; execute the instruction; decode the instruction; copy PC to MAR; increment PC by 1; copy the instruction at the memory location in MAR into MDR.

Chapter 1 · Question 3 · Adapted from GitHub exercises

From source code to execution

Three translator roles

Tool	Input and result
Assembler	Assembly mnemonics and labels become machine instructions.
Compiler	Source program becomes another representation, often native code or bytecode.
Interpreter	Runs program operations through a runtime, sometimes using bytecode.

Choose the right level

Machine code contains the instructions a processor executes. Assembly exposes architecture-specific operations and registers. High-level languages offer abstractions such as variables, functions and data structures. They improve portability, but some translation or runtime support is still needed. A language may have both compiled and interpreted implementations.

Trade-offs are implementation-dependent

Ahead-of-time compilation can support fast execution and catch some errors before running. Interpretation can support rapid experimentation and interactive debugging. Many modern systems combine compilation, interpretation and just-in-time compilation. It is too simple to say that all interpreters translate one source line at a time or that all compilers produce a standalone executable.

PAUSE & TRY

Assemblers, compilers and interpreters are all used to translate or execute programs. Discuss the different roles played by each translator.

Chapter 1 · Question 5 · Adapted from GitHub exercises

Trace a tiny program

Teaching assembly: one instruction per address

Address	Instruction	Meaning
100	LOAD #3	Put the literal value 3 in ACC.
101	ADD #4	Add the literal value 4 to ACC.
102	STORE 200	Copy ACC to memory address 200.
103	HALT	Stop this teaching program.

Watch state change

Start with PC = 100. After LOAD, ACC = 3. After ADD, ACC = 7. STORE copies 7 to address 200; it does not mean "store the number 200". The # sign denotes an immediate value in this invented teaching notation. The example illustrates execution; it is not the instruction set of a particular real CPU.

Two passes and a helpful editor

A two-pass assembler first builds a table connecting labels to addresses, then uses it to resolve operands and encode instructions. An integrated development environment (IDE) combines editing, syntax highlighting, completion, translation and debugging. Breakpoints and single-stepping let you inspect the changing PC, registers and memory.

PAUSE & TRY

Assemblers, compilers and interpreters are all used to translate or execute programs. Discuss the different roles played by each translator.

Chapter 1 · Question 5 · Adapted from GitHub exercises

The operating system's jobs

Coordinate shared resources

Management task	Purpose
Processes	Schedule CPU time and coordinate running programs.
Memory	Allocate and protect memory; manage virtual memory.
Devices and files	Use drivers, organise storage and provide I/O services.
Security	Authenticate users and enforce access permissions.

Utilities support maintenance

Backup software creates recoverable copies; test restoration as well as copying. Antivirus checks for known and suspicious malware activity. A formatter prepares a file system and may destroy access to existing data. Repair tools check file-system structures. Compression tools reduce storage or transfer size. Defragmentation helps some hard disks; it is not routine maintenance for SSDs.

Software does not replace hardware

The OS provides a managed interface to physical resources. A program still needs CPU time, working memory and access to its files. Good resource management improves sharing and responsiveness, but it does not remove limits in memory capacity, processing speed or storage performance.

PAUSE & TRY

File History and Time Machine are examples of backup utilities. Explain why it is important to back up files on a computer.

Chapter 1 · Question 4 · Adapted from GitHub exercises

Events, interrupts & peripherals

Respond without checking everything constantly

An interrupt requests the processor's attention. For an enabled interrupt, the processor and operating system preserve the required execution state, transfer control to an interrupt service routine, then restore state and resume. Exact timing and saved registers depend on the architecture. Not every request is handled immediately; priorities and masking affect when it is serviced.

Interrupt service, conceptually

A timer interrupt helps an OS share CPU time; an input device may interrupt when data becomes available.

Common peripheral connections

Connection	Typical purpose
USB	Serial data, plug-and-play peripherals; many versions also provide power.
HDMI	Digital video and audio for a display.
VGA	Legacy analogue video connection.

PAUSE & TRY

What is a hardware interrupt, and what happens to program execution when an enabled interrupt is serviced by the CPU?

Chapter 3 · Question 6 · Adapted from GitHub exercises

Key terms · 01

Build your vocabulary

Term	Meaning
Bit / byte	A binary digit / a group of eight bits.
Binary / denary / hexadecimal	Base-2 / base-10 / base-16 number systems.
Two's complement	A fixed-width signed representation with a negative most-significant-bit weight.
BCD	Four bits encode each individual decimal digit.
Character set / encoding	Available characters / how those characters are represented as data.
Resolution / colour depth	Pixel dimensions or density / bits used per pixel.
Sampling rate / bit depth	Samples per second / bits per sample.
Lossless / lossy	Exactly reversible / information-discarding compression.
Stored program	Instructions and data held in memory for execution.

Use the terms in an explanation

Choose two terms and connect them in a sentence about a working system. A definition is useful when it helps you explain a mechanism, compare alternatives or justify a decision. Return to the relevant teaching page if you cannot yet give an example.

PAUSE & TRY

Give one use of the binary-coded decimal system.

Chapter 1 · Question 1c · Adapted from GitHub exercises

Key terms · 01

Build your vocabulary

Term	Meaning
ALU / control unit	Arithmetic and logic / coordination of operations.
PC / MAR / MDR	Next instruction address / accessed address / data being transferred.
CIR / ACC	Current instruction / intermediate arithmetic result in the teaching model.
Address / data / control bus	Location information / values and instructions / coordination signals.
Compiler / interpreter / assembler	Different mechanisms for translating or running programs.
Interrupt / context	A request for attention / the state needed to resume execution.
IDE	An integrated environment for editing, running and debugging code.

Use the terms in an explanation

Choose two terms and connect them in a sentence about a working system. A definition is useful when it helps you explain a mechanism, compare alternatives or justify a decision. Return to the relevant teaching page if you cannot yet give an example.

PAUSE & TRY

Give one use of the binary-coded decimal system.

Chapter 1 · Question 1c · Adapted from GitHub exercises

Practice · Chapter 1

Try first, then check feedback

Write your reasoning before opening the explained answers after the six chapter missions. Show units and intermediate steps for calculations. For comparisons, use the same criteria on both sides. Match the question number with the answer key in this complete edition.

Chapter 1 · Question 1a2

The byte 10011010 represents an eight-bit two's complement integer. State its equivalent denary value.

Chapter 1 · Question 1a4

State the range of integers that can be represented by an eight-bit two's complement number. Give the endpoints in denary.

Chapter 1 · Question 1b1

Write the denary integer 798 in binary-coded decimal (BCD) format.

PAUSE & TRY

Give one use of the binary-coded decimal system.

Chapter 1 · Question 1c · Adapted from GitHub exercises

Practice · Chapter 1

Try first, then check feedback

Write your reasoning before opening the explained answers after the six chapter missions. Show units and intermediate steps for calculations. For comparisons, use the same criteria on both sides. Match the question number with the answer key in this complete edition.

Chapter 1 · Question 2a1

A developer records sounds for a game. State what is meant by sampling resolution.

PAUSE & TRY

A developer includes bitmap images in a game. Explain the term image resolution.

Chapter 1 · Question 2b1 · Adapted from GitHub exercises

MORNING

BEHAVIORISM & BEHAVIORIST AI

2

Learning from the environment

Start with observable behaviour

Behaviourism studies the relationship between observable behaviour and environmental events. A stimulus is a detectable event; a response is an observable reaction. Pavlov, Watson and Skinner contributed to different strands of this tradition. The approach helped make learning measurable, although it is not a complete description of every kind of human learning.

Two kinds of conditioning

Classical conditioning	Operant conditioning
A cue becomes associated with another stimulus.	An action becomes associated with its consequences.
A bell paired with food can trigger salivation.	Sitting followed by a treat can become more frequent.
Focus: an elicited response.	Focus: behaviour shaped by reinforcement or punishment.

Connect the analogy carefully

Reinforcement learning borrows the idea of learning from consequences, but an algorithm is not a human or animal mind. A numerical reward is a training signal. It does not imply feelings, understanding or consciousness. Use the analogy to identify a feedback relationship, then describe the computational mechanism precisely.

PAUSE & TRY

Differentiate between classical conditioning (Pavlov) and operant conditioning (Skinner) in terms of how learning occurs.

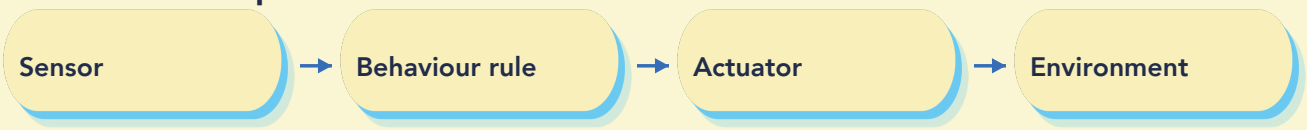
Chapter 2 · Question 1 · Adapted from GitHub exercises

React first: behaviour-based robots

A sensor can trigger a useful action

A simple robotic vacuum may move forward until a bumper is pressed, then reverse and turn. Its environment is the room, its stimulus is the bumper signal, and its response is the movement. These rules can produce useful behaviour without a detailed map. If the rules never change from experience, this is reactive control rather than reinforcement learning.

A direct feedback path



Situated systems act in an environment; embodied systems have a physical body that constrains and enables action.

Layers and priorities

Brooks' subsumption architecture coordinates relatively simple behaviour modules. A module can inhibit or suppress another module's output under specified conditions. A designer might combine wandering, obstacle avoidance and task seeking while protecting a safety stop. This is one design approach, not a claim that all map-based planning is slow or unreliable; practical robots often combine reactive control and planning.

PAUSE & TRY

Explain how subsumption architecture organises robot behaviours into layered modules without requiring a centralised internal map.

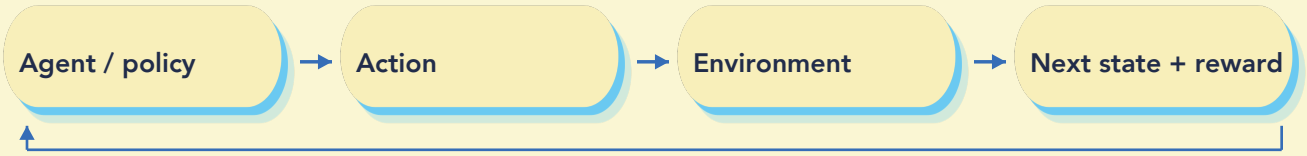
Chapter 2 · Question 3 · Adapted from GitHub exercises

The reinforcement-learning loop

Learn a policy through experience

Reinforcement learning is a way for an agent to learn decisions through interaction. The agent observes a state or observation, selects an action, and receives feedback from the environment. A learning algorithm uses that experience to improve a policy: the rule or strategy for choosing actions. The objective is usually expected cumulative reward, not just the next reward.

One interaction at time t



Repeat: observe, act, receive feedback, then update. The learning update may occur during training rather than every deployed action.

Name the components

Component	Grid-world example
Agent / environment	A simulated rover / a grid with obstacles.
State / action	Current square / move up, down, left or right.
Reward / policy	Points for outcomes / a strategy for selecting moves.

PAUSE & TRY

What are the roles of the environment, the agent and the policy in a Markov decision process (MDP)?

Chapter 2 · Question 4a · Adapted from GitHub exercises

Rewards, policies & values

Feedback evaluates an outcome

A reward is a numerical signal associated with an interaction. It may be positive, zero or negative. A reward of −1 is not a moral judgement: its role is defined by the task. Designers might reward reaching a goal, penalise a collision and charge a small cost per step. Poorly chosen rewards can encourage shortcuts that defeat the intended goal.

Distinguish the three ideas

The reward says how an outcome scores now. A policy says what action to choose in a situation. A value estimate says how much future reward is expected from a state or a state–action choice. A good policy may accept a small immediate cost to achieve a larger long-term return. Q-learning estimates action values and derives choices from them.

The learning question

Experience → **Value estimate** → **Policy improvement**

A fixed rule such as "if hot, turn on fan" may control a system effectively without learning any values or changing a policy.

PAUSE & TRY

Explain how an RL-based system uses feedback or reward signals from its environment to update its behavioural policy or value estimates.

Chapter 5 · Question 3a · Adapted from GitHub exercises

Explore or exploit?

Use the restaurant analogy

Exploitation is ordering a dish you already believe is best. Exploration is trying another dish to learn more about it. In an RL task, exploitation chooses the action with the best current value estimate, including future return. Exploration gathers information about other actions. Early estimates can be wrong, so always exploiting can lock an agent into an inferior policy.

An epsilon-greedy example

Choice rule	With epsilon = 0.10
Explore	With 10% probability, choose a random permitted action.
Exploit	Otherwise choose an action with the highest current Q value.
During training	Measure outcomes and update estimates; epsilon may change over time.

Think before choosing

Suppose one route has worked twice and another has never been tried. Is the first route certainly better? No: evidence is incomplete. Exploration can improve knowledge, but it also has a cost. In physical systems, experiments must respect safety constraints; training in a simulation does not automatically make every real-world action safe.

PAUSE & TRY

Explain the difference between exploration (trying new actions) and exploitation (using the best current estimates). Why must an agent balance both?

Chapter 2 · Question 4b · Adapted from GitHub exercises

Update a Q value

A table of learned action values

Q(s, a) estimates the return from taking action a in state s and then following a good policy. Q-learning is model-free: it learns from transitions without needing a supplied transition model. "Model-free" does not mean memory-free or that the agent lacks a table or neural network.

The update rule

Q(new) = Q(old) + alpha × [reward + gamma × max Q(next state, next action) − Q(old)]. Alpha controls the size of an update. Gamma discounts future rewards. The quantity in square brackets is the temporal-difference error. For a terminal next state, use zero for the future-value term.

Work through one illustrative update

Quantity	Value
Old Q / alpha	2 / 0.5
Reward / gamma / best next Q	1 / 0.9 / 4
Target	1 + 0.9 × 4 = 4.6
New Q	2 + 0.5 × (4.6 − 2) = 3.3

Interpret the result

This experience raised the estimate from 2 to 3.3. It did not prove the action is always best. Repeated experience, an appropriate learning schedule and sufficient exploration are important; one reward does not establish a reliable policy.

PAUSE & TRY

Explain the difference between exploration (trying new actions) and exploitation (using the best current estimates). Why must an agent balance both?

Chapter 2 · Question 4b · Adapted from GitHub exercises

A decision problem as an MDP

Five parts of a Markov decision process

Symbol	Meaning
S	Possible states.
A	Available actions.
P	Transition probabilities: where an action can lead.
R	Reward function: how outcomes are scored.
gamma	Discount factor: how future rewards are weighted.

The Markov property

A state is Markov when it contains the information needed to predict the next-state and reward distribution, given the current action. Earlier history adds no further predictive information once that state is known. A position alone may be insufficient: a robot may also need speed, battery level or whether it is carrying a package.

Observation is not always state

Sensors can provide incomplete or noisy observations. A camera view may hide an obstacle or omit velocity. An agent may need memory or a state estimate to work with partial information. A simple grid world is useful because we can explicitly specify the state, actions, transitions and rewards instead of assuming the agent knows everything.

PAUSE & TRY

What are the roles of the environment, the agent and the policy in a Markov decision process (MDP)?

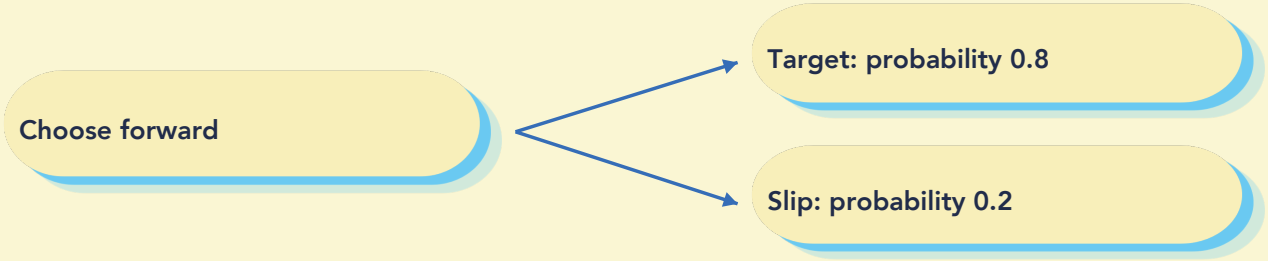
Chapter 2 · Question 4a · Adapted from GitHub exercises

Actions can have uncertain outcomes

A slippery grid square

Imagine an agent choosing “forward”. In this illustrative environment, it reaches the intended square with probability 0.8 and slips sideways with probability 0.2. The action is chosen by the agent; the transition outcome is sampled by the environment. The probabilities for all possible next outcomes must sum to 1.

One action, two outcomes



For immediate rewards of +10 and −5, the expected immediate reward is 0.8 × 10 + 0.2 × (−5) = 7. Future rewards may change which action is best.

Expected is not guaranteed

An expected value of 7 does not mean every attempt earns 7. It summarises the average implied by the model. A system that learns under one set of transition probabilities may perform poorly when its environment changes. Test across plausible conditions rather than judging a policy from a single successful run.

PAUSE & TRY

What are the roles of the environment, the agent and the policy in a Markov decision process (MDP)?

Chapter 2 · Question 4a · Adapted from GitHub exercises

Training is different from using a policy

Separate two phases

During training, the system collects experience and updates estimates or policy parameters. During deployment, it may use a fixed learned policy without further learning. Some applications allow controlled online updates, but adaptation is not automatic. A robot can react to a new sensor reading while keeping exactly the same policy.

Test what was actually learned

Check whether success persists with different starting states, sensor noise and changed conditions. Compare against a simple fixed-rule baseline. Examine failure cases as well as average reward. If the goal is "arrive safely", a reward that only measures speed may produce an undesirable strategy even when the algorithm is working as specified.

A responsible learning workflow

Define the task → **Train / explore** → **Evaluate** → **Deploy / monitor**

Behaviour-based robotics, reinforcement learning and neural networks are related ideas, not interchangeable labels. A system can use one without using all three.

PAUSE & TRY

Give an example of a simple reflex-based agent, such as a robotic vacuum or thermostat, and identify its stimulus, response and environment.

Chapter 2 · Question 2 · Adapted from GitHub exercises

Key terms · 02

Build your vocabulary

Term	Meaning
Behaviourism	An approach studying observable behaviour and its relation to the environment.
Stimulus / response	A detectable event / the observable reaction.
Subsumption	Layered behaviour coordination using suppression or inhibition.
Agent / environment	The decision-maker / the system it interacts with.
State / observation	A representation sufficient for a task / the information actually sensed.
Action / reward	A choice / a numerical feedback signal.
Policy	A strategy that maps a situation to action choices.
Value / Q value	Expected return from a state / from a state–action choice.
Exploration / exploitation	Gathering new experience / using current value estimates.

Use the terms in an explanation

Choose two terms and connect them in a sentence about a working system. A definition is useful when it helps you explain a mechanism, compare alternatives or justify a decision. Return to the relevant teaching page if you cannot yet give an example.

PAUSE & TRY

Give an example of a simple reflex-based agent, such as a robotic vacuum or thermostat, and identify its stimulus, response and environment.

Chapter 2 · Question 2 · Adapted from GitHub exercises

Key terms · 02

Build your vocabulary

Term	Meaning
Learning rate	A parameter controlling the size of an update.
Discount factor	A parameter controlling the weighting of future rewards.
MDP	A framework specifying states, actions, transitions, rewards and discounting.
Model-free RL	Learning without a supplied environment transition model.
Markov property	The current state and action contain the information needed for the next-step distribution.

Use the terms in an explanation

Choose two terms and connect them in a sentence about a working system. A definition is useful when it helps you explain a mechanism, compare alternatives or justify a decision. Return to the relevant teaching page if you cannot yet give an example.

PAUSE & TRY

Give an example of a simple reflex-based agent, such as a robotic vacuum or thermostat, and identify its stimulus, response and environment.

Chapter 2 · Question 2 · Adapted from GitHub exercises

MIDDAY

HARDWARE

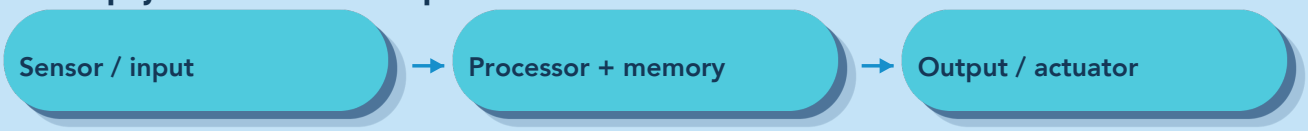
3

Hardware gives software a body

Classify by the job being performed

Hardware is the physical part of a computer system. An input device supplies data, a processing device transforms it, an output device communicates or acts on a result, and storage retains information. One device can have several roles: a touchscreen displays output and senses input. A barcode reader is input; an SSD is storage.

From a physical event to a response



The processor follows instructions. Useful behaviour depends on the program, the data, the hardware and their connections.

How hardware enables AI

An AI model must run as computations on hardware. Sensors provide observations, memory holds model parameters and working data, a processor evaluates the model, and an output device communicates the decision. A physical robot also needs a powered actuator to move. More powerful hardware can increase capacity or speed, but does not itself create a learning algorithm.

PAUSE & TRY

Classify the following as input, output or storage, noting dual roles where relevant: a touchscreen display, a solid-state drive and a barcode reader.

Chapter 3 · Question 3 · Adapted from GitHub exercises

Memory: capacity versus speed

A useful hierarchy

Resource	Role and trade-off
Registers	Very small, very fast storage inside the processor.
Cache	Fast memory holding copies of recently or frequently used data.
Main RAM	Working storage for active programs and data; normally volatile.
Secondary storage	Larger, persistent storage; usually slower to access.

SRAM and DRAM

Static RAM stores bits using bistable circuits such as flip-flops. It needs power but no periodic refresh, and is commonly used for CPU caches. Dynamic RAM stores charge in cells that must be refreshed; its greater density and lower cost make it suitable for main memory. Both are volatile. Cache can be on the processor chip; it is not necessarily external.

Why an AI workload needs memory

Model weights, input data and intermediate results must fit somewhere. A fast processor can spend time waiting for data if memory bandwidth is insufficient. Capacity answers "how much fits?" Bandwidth answers "how quickly can it move?" These are different constraints and both affect performance.

PAUSE & TRY

Explain the structural difference between SRAM and DRAM regarding how they store data and their need for refreshing.

Chapter 3 · Question 2a · Adapted from GitHub exercises

Keep data when power is off

ROM and flash families

Type	Writing or erasing
Mask ROM	Programmed during manufacture.
PROM	Programmed once after manufacture.
EPROM	Erased with ultraviolet light, then reprogrammed.
EEPROM	Electrically erasable and reprogrammable.
Flash	Electrically erasable memory, normally erased in blocks.

Magnetic, solid-state and optical media

A hard disk stores data magnetically on rotating platters. Seek time and rotational latency delay access; fragmentation can increase head movement. An SSD stores data in flash with no moving parts. It avoids mechanical seek and rotation delays, but still has nonzero access latency and finite write endurance. External drives can provide portable backup storage.

Optical storage

CDs, DVDs and Blu-ray discs use laser light to read recorded features. Some discs have multiple recording layers. Material defects and optical effects, including birefringence, can disrupt reading. Choose a medium using capacity, access time, cost, durability and the need for long-term retention rather than assuming one technology is best for every purpose.

PAUSE & TRY

Contrast ROM, PROM and EPROM in terms of how data is written and erased.

Chapter 3 · Question 2b · Adapted from GitHub exercises

Processor instruction-set choices

Introductory design tendencies

RISC	CISC
Regular, often simpler instruction encodings.	Can include more complex and variable-length instructions.
Typically separates loads/stores from register arithmetic.	May allow operations with memory operands.
Examples include ARM and RISC-V families.	A major example is the x86 family.

The distinction is not a speed guarantee

A reduced instruction set does not guarantee that every instruction takes one clock cycle. A complex instruction set does not prevent pipelining or high performance. Modern processors use caches, pipelines, prediction and other techniques; many translate instructions internally into simpler operations. Some RISC families also support compressed instruction encodings.

Look at the workload

Performance depends on implementation, clock frequency, instructions per cycle, memory behaviour, software and energy constraints. A mobile device may prioritise power efficiency; a workstation may prioritise throughput. Compare measured work done under relevant conditions instead of judging a CPU from the RISC or CISC label alone.

PAUSE & TRY

Explain the main design difference between RISC and CISC architectures regarding instruction encoding and decoding complexity.

Chapter 3 · Question 5a · Adapted from GitHub exercises

Overlap work with a pipeline

Use the laundry analogy

A laundry can wash one load while drying an earlier load. Similarly, a processor pipeline can fetch a new instruction while decoding or executing previous ones. This increases throughput after the pipeline fills; it does not necessarily reduce the latency of an individual instruction. Pipelining is used in both RISC and CISC implementations.

A simplified four-stage pipeline

Instruction	Cycle 1	2	3	4	5
I1	Fetch	Decode	Execute	Write	—
I2	—	Fetch	Decode	Execute	Write
I3	—	—	Fetch	Decode	Execute

Why a pipeline can pause

A data hazard occurs when an instruction needs a result that is not ready. A control hazard occurs when a branch makes the next instruction uncertain. A structural hazard occurs when operations need the same resource. Stalling, forwarding, scheduling and prediction can help, depending on the processor. "Overlapping" does not mean every instruction is independent.

PAUSE & TRY

Explain the role of pipelining in overlapping instruction execution, and name one hazard that can stall a pipeline.

Chapter 3 · Question 5b · Adapted from GitHub exercises

One stream or many?

Flynn's classification

Class	Instruction / data streams	Example idea
SISD	Single / single	One instruction stream processes individual data.
SIMD	Single / multiple	One operation applied to many array elements.
MISD	Multiple / single	Rare; different processing of one data stream.
MIMD	Multiple / multiple	Independent cores or computers run different work.

Scale beyond one processor

A cluster connects computers, called nodes, so they can coordinate work. Massively parallel systems distribute large tasks across many processing units. Supercomputers are designed for demanding scientific or engineering computation; they are not simply a synonym for mainframes. Communication, synchronisation and data movement affect whether adding processors actually helps.

Connect this to AI

Large arrays of multiplications can benefit from parallel hardware. Parts of an AI workload may still be sequential or limited by memory and communication. A GPU has its own execution model, often described using SIMT, and provides SIMD-like parallel arithmetic; the simple taxonomy is a starting point, not a complete description of every chip.

PAUSE & TRY

Explain the role of pipelining in overlapping instruction execution, and name one hazard that can stall a pipeline.

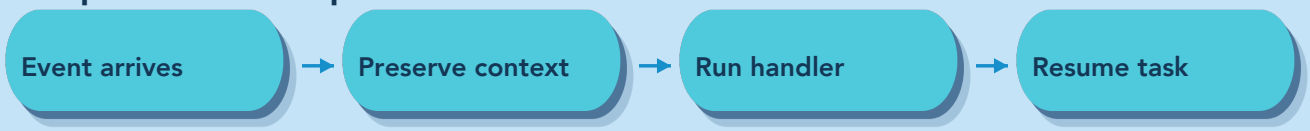
Chapter 3 · Question 5b · Adapted from GitHub exercises

Let hardware request attention

Events can change the schedule

A sensor, network interface or timer can raise an interrupt. The CPU responds when the request is enabled and eligible. It preserves enough context to resume the interrupted work and transfers execution to a handler. The handler deals with the event and returns. The hardware and operating system divide the responsibility for saving and restoring state.

A simplified service sequence



Context can include the program counter, status and required registers. Exact details vary by architecture.

Predictability matters

A system with a timing deadline must account for interrupt priority, handler execution time and other sources of delay. RISC versus CISC alone does not determine interrupt latency. Long or frequent interrupt handling can delay useful work. A well-designed system measures worst-case behaviour rather than assuming every event will receive an immediate response.

PAUSE & TRY

What is a hardware interrupt, and what happens to program execution when an enabled interrupt is serviced by the CPU?

Chapter 3 · Question 6 · Adapted from GitHub exercises

Reason with logic gates

Gates transform binary signals

AND produces 1 only when all inputs are 1. OR produces 1 when at least one input is 1. NOT inverts one input. XOR produces 1 when two inputs differ. NAND is the inverse of AND; NOR is the inverse of OR. A truth table lists every possible input combination and its corresponding output.

Two-input truth table

A	B	AND	OR	XOR	NAND	NOR
0	0	0	0	0	1	1
0	1	0	1	1	1	0
1	0	0	1	1	1	0
1	1	1	1	0	0	0

Read symbols and conditions

NOT maps 0 to 1 and 1 to 0. An AND symbol has a flat input side and curved output side; an OR symbol has a curved input side and pointed output. A small output circle means inversion. In a control rule, “door closed AND start pressed” requires both conditions; “smoke OR manual alarm” requires either.

PAUSE & TRY

Draw or describe the logic-gate symbols and write the truth tables for AND, OR and NOT.

Chapter 3 · Question 7 · Adapted from GitHub exercises

Simplify a logic expression

Useful identities

Law	Equivalent expressions
Identity	A AND 1 = A; A OR 0 = A
Null	A AND 0 = 0; A OR 1 = 1
Idempotent	A AND A = A; A OR A = A
Complement	A AND NOT A = 0; A OR NOT A = 1
De Morgan	NOT(A AND B) = (NOT A) OR (NOT B)
De Morgan	NOT(A OR B) = (NOT A) AND (NOT B)

A worked simplification

Let X = (A AND B) OR (A AND NOT B). Factor out A: X = A AND (B OR NOT B). The term in brackets is always 1, so X = A AND 1 = A. A simpler equivalent circuit can reduce hardware and delay. Check equivalence by comparing truth-table outputs for every input combination.

Keep the negation in the right place

De Morgan's laws negate the whole grouped expression on the left. Dropping that outer NOT changes the meaning. Parentheses make the scope clear, especially when a printed overbar is difficult to see. In Boolean algebra, OR is not ordinary arithmetic addition: 1 OR 1 is still 1.

PAUSE & TRY

Draw or describe the logic-gate symbols and write the truth tables for AND, OR and NOT.

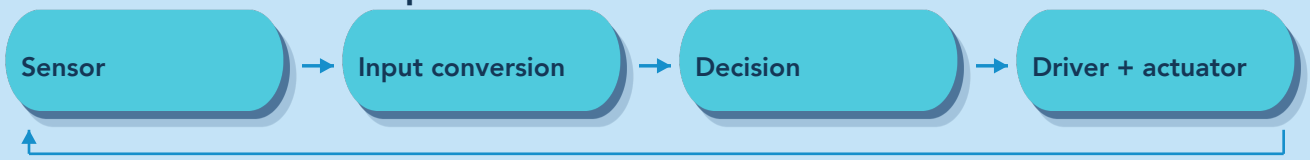
Chapter 3 · Question 7 · Adapted from GitHub exercises

Sense, decide, act, sense again

Monitoring versus control

A monitoring system records or displays measurements. A control system also acts on the environment. In a greenhouse, a temperature sensor supplies a reading, a processor compares it with a target, and a fan changes the temperature. A later measurement closes the feedback loop. This can be a fixed-rule controller; learning is not required.

An environmental control loop



The changed environment is measured again. Analogue sensors may need an ADC; many sensors already provide digital data.

Signals are not actuator power

An ADC converts an analogue signal to a digital value. A DAC converts a digital value to an analogue signal. Some actuators use PWM or digital interfaces instead of a DAC. PWM is a timed switching signal, not the same thing as analogue conversion. A motor driver or power stage supplies the current needed to move a motor or operate a valve.

PAUSE & TRY

Outline an automated environmental control system, such as a greenhouse or thermostat. Specify the sensor, the processing condition and the resulting actuator action.

Chapter 3 · Question 4 · Adapted from GitHub exercises

Computing inside another device

A dedicated purpose

An embedded system performs a particular role within a larger product, such as a washing-machine controller or a sensor node. A microcontroller typically combines a CPU, memory and input/output peripherals on one chip. An embedded system may use a microcontroller, a larger processor or specialised hardware, depending on its requirements.

Benefits and constraints

Potential benefit	Constraint to consider
Compact, low-power operation.	Limited memory, processing capacity and battery energy.
Low cost for repeated tasks.	Upgrades and repairs may be difficult.
Predictable behaviour with careful design.	Timing and reliability must be tested, not assumed.
Local operation without a cloud link.	External data or remote management may still need a network.

Design around the task

A small fixed-rule controller may need only modest hardware. A vision model can require much more memory and computation. Choose sensors, processing capacity, interfaces and power together. Environmental temperature, moisture, vibration and expected service life also influence whether a design is suitable.

PAUSE & TRY

Explain how thermal limits, power consumption, memory capacity and processing speed constrain complex AI models on small embedded devices.

Chapter 6 · Question 1 · Adapted from GitHub exercises

The hardware behind an AI model

Different processing strengths

Hardware	Useful role
CPU	General control, branching and diverse program tasks.
GPU	High-throughput parallel arithmetic for suitable workloads.
NPU / TPU-type accelerator	Specialised tensor operations with hardware-specific efficiency.
RAM and storage	Hold weights, inputs, intermediate results and saved models.

Training and inference have different costs

Training adjusts model parameters using data or experience. Inference uses a model to produce an output. A model may be trained on a large server and deployed on a smaller device. Simple tabular reinforcement learning can run on an ordinary CPU; AI does not always require a GPU, and a specialised chip is not automatically faster for every workload.

Trace the whole path

For a camera-based robot, the sensor captures an image, memory buffers it, processing estimates a useful state, a policy selects an action, and control hardware drives the actuator. Delays in any stage matter. Model size, memory movement, power consumption and heat can be as important as the advertised arithmetic speed of the chip.

PAUSE & TRY

Explain how thermal limits, power consumption, memory capacity and processing speed constrain complex AI models on small embedded devices.

Chapter 6 · Question 1 · Adapted from GitHub exercises

Interfaces you can see and touch

Touch and display technologies

Technology	Key idea
Capacitive touch	Detects changes in an electric field; often supports multi-touch.
Resistive touch	Pressure brings conductive layers into contact.
OLED display	Organic light-emitting pixels produce their own light.
VR headset	Combines displays, optics and tracking to present an immersive view.

Display quality involves trade-offs

Screen resolution describes pixel dimensions. OLED can provide deep blacks because individual pixels can be off, but power use depends on displayed content and hardware. Capacitive and resistive touch differ in input methods, clarity, durability and cost. A touchscreen is both an input surface and an output display.

Build physical objects layer by layer

In material-extrusion 3D printing, a nozzle deposits material to form successive layers. In binder-based printing, a binder joins selected regions of a powder bed. These are different additive-manufacturing methods; material properties and post-processing requirements differ. A printer turns digital instructions into physical output through controlled motion and material handling.

PAUSE & TRY

Classify the following as input, output or storage, noting dual roles where relevant: a touchscreen display, a solid-state drive and a barcode reader.

Chapter 3 · Question 3 · Adapted from GitHub exercises

Key terms · 03

Build your vocabulary

Term	Meaning
Hardware	Physical computing components.
Input / output / storage	Data entry / results or action / retention of information.
Volatile / non-volatile	Loses data without power / retains data without power.
SRAM / DRAM	Bistable-cell memory / charge-based memory requiring refresh.
ROM / PROM / EPROM	Read-only / one-time programmable / ultraviolet-erasable memory.
EEPROM / flash	Electrically erasable memory / a block-erasable family.
Cache	Fast storage holding copies of data near a processor.
HDD / SSD / optical	Magnetic disk / solid-state storage / laser-readable media.
RISC / CISC	Instruction-set design families; not direct performance guarantees.

Use the terms in an explanation

Choose two terms and connect them in a sentence about a working system. A definition is useful when it helps you explain a mechanism, compare alternatives or justify a decision. Return to the relevant teaching page if you cannot yet give an example.

PAUSE & TRY

Differentiate between primary storage and secondary storage in terms of volatility, speed and CPU accessibility.

Chapter 3 · Question 1a · Adapted from GitHub exercises

Key terms · 03

Build your vocabulary

Term	Meaning
Pipeline / hazard	Overlapped instruction stages / a dependency or conflict that can stall them.
SISD / SIMD / MISD / MIMD	Flynn's categories of instruction and data streams.
Sensor / actuator	Measures a physical condition / produces physical action.
ADC / DAC / PWM	Analogue-to-digital conversion / digital-to-analogue conversion / pulse-width modulation.
Microcontroller	An integrated processor with memory and input/output peripherals.
Embedded system	A computing system with a role inside a larger product.
GPU / NPU	Parallel graphics-oriented processor / specialised neural-processing hardware.
Logic gate / truth table	A binary operation circuit / its outputs for every input combination.

Use the terms in an explanation

Choose two terms and connect them in a sentence about a working system. A definition is useful when it helps you explain a mechanism, compare alternatives or justify a decision. Return to the relevant teaching page if you cannot yet give an example.

PAUSE & TRY

Differentiate between primary storage and secondary storage in terms of volatility, speed and CPU accessibility.

Chapter 3 · Question 1a · Adapted from GitHub exercises

Practice · Chapter 3

Try first, then check feedback

Write your reasoning before opening the explained answers after the six chapter missions. Show units and intermediate steps for calculations. For comparisons, use the same criteria on both sides. Match the question number with the answer key in this complete edition.

Chapter 3 · Question 1a

Differentiate between primary storage and secondary storage in terms of volatility, speed and CPU accessibility.

Chapter 3 · Question 1b

Give an example of a secondary-storage technology for each category: magnetic, optical and solid-state (flash).

PAUSE & TRY

State the functions of the program counter, accumulator and memory data register during instruction execution.

Chapter 3 · Question 5c · Adapted from GitHub exercises

AFTERNOON

NETWORKING

4

Connect a room, a city, a world

What a network provides

A network connects devices so they can exchange data and share services, storage or hardware. A connected device is a node. Networks differ in scale, ownership, transmission media and management. Geographic size alone does not determine speed: a particular WAN link can be faster than a particular local wireless connection.

Names describe the scope

Network	Typical scope
PAN / WPAN	A person's nearby devices; a WPAN is wireless.
LAN / WLAN	A home, office or building; a WLAN uses wireless links.
MAN	A metropolitan area or several sites across a city.
WAN	Widely separated sites, regions or countries.

A practical analogy

A personal conversation resembles a PAN, a building intercom resembles a LAN, and an inter-city transport network resembles a WAN. Real networks can contain smaller networks inside them. The Internet is a global interconnection of networks, built from links and equipment operated by many organisations.

PAUSE & TRY

Contrast LANs and WANs in geographic scale, ownership and typical transmission conditions. Avoid assuming that scale alone fixes speed.

Chapter 4 · Question 1a · Adapted from GitHub exercises

Clients, servers & peers

Two ways to organise services

Client–server	Peer-to-peer
Clients request services from servers.	Peers can both request and provide resources.
Central management can simplify permissions and backup.	Resources can be distributed among participating peers.
Capacity and resilience require deliberate design.	Coordination and consistent security can be harder.

Servers are roles, not magic boxes

A file server provides access to shared files. One physical machine can host several server programs, and a service can run across many machines. A server failure need not stop an entire network if replication or failover is provided. A small P2P setup can be inexpensive, while large P2P systems can also scale when their protocols are designed for it.

A service request

Client request → **Network** → **Server response**

A peer may act as a client for one exchange and as a server for another. Describe the role being performed.

PAUSE & TRY

Compare the client–server and peer-to-peer models in resource management and scalability.

Chapter 4 · Question 1b · Adapted from GitHub exercises

Where does the work happen?

Thin and thick clients

Thin client	Thick client
Relies heavily on remote applications or services.	Performs substantial processing locally.
Can simplify central administration.	Can support more offline work.
Network and server availability affect its main tasks.	Needs suitable local hardware and maintenance.

Public and private clouds

A public cloud provides services using infrastructure operated by a provider for multiple customers. A private cloud is dedicated to one organisation and can be on-site or hosted elsewhere. Public cloud can reduce initial capital cost and offer flexible capacity; private cloud can provide greater direct control but requires resources to manage.

Compare actual responsibilities

Neither deployment is automatically secure. Consider access control, configuration, backups, data location, operational expertise and ongoing cost. Replication improves availability but is not a substitute for a recoverable backup. A thin client still performs some local processing, such as displaying an interface; “thin” does not mean it has no CPU.

PAUSE & TRY

What are the main trade-offs in cost, security responsibilities and control between a public cloud and a private cloud?

Chapter 4 · Question 1d · Adapted from GitHub exercises

The devices along a network path

Recognise the job

Device	Primary role
NIC	Connects a host to a network interface.
Switch / bridge	Forwards frames between LAN ports or segments.
Router	Forwards IP packets between networks.
Wireless access point	Connects wireless stations to a network.
Hub / repeater	Repeats signals; a hub sends them to multiple ports.
Gateway / modem	Translates between systems / modulates and demodulates signals.

Addresses serve different scopes

An Ethernet switch learns which MAC addresses are reachable through which ports. A router uses IP routing information to select a next hop. MAC addresses can be changed or randomised; they are not always permanently fixed identifiers. A modem's modulation depends on the medium, rather than being limited to old telephone lines.

A switch is selective, not omniscient

Known unicast frames can be sent to a selected port. Broadcasts and unknown destinations may be flooded within the relevant LAN. A router chooses a route using its routing table and policies; it does not always measure the least congested possible path for every packet.

PAUSE & TRY

Outline the four TCP/IP layers in order and give the core responsibility of each.

Chapter 4 · Question 4a · Adapted from GitHub exercises

Sharing a communication medium

Choose a link for its conditions

Twisted-pair copper reduces some electromagnetic interference by twisting paired conductors. Optical fibre carries light and supports high-capacity links. Wi-Fi uses shared radio channels; Bluetooth connects nearby devices using its own radio protocols. Range, speed and frequency-hopping behaviour depend on the version, equipment and environment, not one universal distance or channel count.

Collision detection and avoidance

CSMA/CD	CSMA/CA
Historically used on shared, half-duplex Ethernet.	Used by Wi-Fi to manage shared radio access.
Listen, transmit, detect collision, back off and retry.	Listen, wait/back off, transmit and use acknowledgements.
Modern switched full-duplex Ethernet has no collisions on the link.	RTS/CTS is an optional mechanism, not a compulsory step every time.

Use the conversation analogy

At a busy table, people listen before speaking and pause if they talk over each other. Random backoff reduces repeated clashes. Avoidance reduces the chance of a collision; it cannot guarantee one never happens. Interference, hidden stations and missing acknowledgements can still cause retries and delay.

PAUSE & TRY

How do network systems handle simultaneous transmissions on a shared medium, using collision detection or collision avoidance?

Chapter 4 · Question 2 · Adapted from GitHub exercises

Internet, web & addresses

The web is one Internet service

The Internet carries many kinds of communication. The World Wide Web uses linked resources accessed through web protocols. Email and other services also use the Internet. IPv4 addresses have 32 bits; IPv6 addresses have 128 bits. An address helps identify a network interface for communication, while routing determines how packets are forwarded.

Read a URL: https://example.org:443/course/notes?q=ai#part2

Part	Meaning
https	Scheme: how the resource is accessed.
example.org / :443	Host name / port number.
/course/notes	Path interpreted by the server.
?q=ai	Query supplied to the application.
#part2	Fragment identifying a location within the resource.

A URL is not just an IP address

The host name may be resolved through DNS. The path selects a resource on that host and need not correspond to a physical directory. Default ports can be omitted. A browser normally handles the fragment locally rather than sending it as part of an HTTP request.

PAUSE & TRY

Break down the parts of a URL, such as scheme, domain and path, and explain how it identifies a resource.

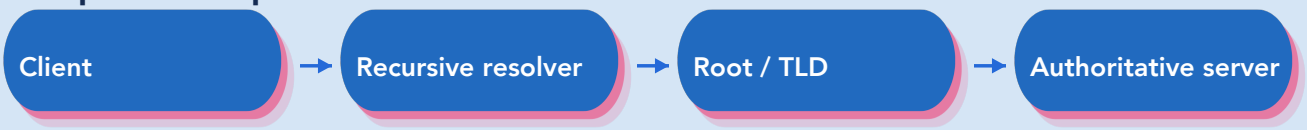
Chapter 4 · Question 3a · Adapted from GitHub exercises

Find a host with DNS

Names are easier to remember

The Domain Name System maps names to records, including records that provide IP addresses. A browser and operating system may already have a cached answer. If not, a recursive resolver can obtain an answer using the DNS hierarchy. Cached answers remain usable for their permitted lifetime, reducing repeat lookups.

A simplified lookup



The resolver follows referrals, receives the requested record, caches it and returns the answer to the client.

The lookup is only one step

After resolving a host, a client still needs to establish suitable communication with the service. DNS does not deliver the web page itself, and it does not translate an entire URL, including its path, into an address. Failures in DNS, connectivity, transport, security negotiation or the application can each prevent a page from loading.

PAUSE & TRY

Describe the role of DNS when resolving the host name in a URL to an IP address.

Chapter 4 · Question 3b · Adapted from GitHub exercises

Give each protocol layer a job

The four-layer TCP/IP model

Layer	Responsibility	Examples
Application	Application messages and services.	HTTP, DNS, SMTP
Transport	Communication between application endpoints.	TCP, UDP
Internet	IP addressing and routing between networks.	IPv4, IPv6, ICMP
Link	Local network delivery and access to the medium.	Ethernet, Wi-Fi

Encapsulation is packaging

Application data is carried inside transport data, which is carried inside IP packets, which are carried by link-layer frames. Each layer has its own fields and scope. A frame's MAC addresses are not the same thing as an IP packet's addresses; the frame can change at the next link while the packet continues its journey.

Reliable ordering is a service choice

TCP provides an ordered byte stream with reliability and flow/congestion control. UDP provides datagrams without those guarantees by itself. Applications may build additional mechanisms over UDP. Not every IP packet contains a sequence number that allows a router to rebuild a file: reconstruction belongs to the relevant endpoint protocol.

PAUSE & TRY

Outline the four TCP/IP layers in order and give the core responsibility of each.

Chapter 4 · Question 4a · Adapted from GitHub exercises

Web, files, email & sharing

Identify the application protocol

Protocol	Purpose
HTTP / HTTPS	Web requests and responses; HTTPS protects HTTP using TLS.
FTP	Traditional file transfer; plain FTP is not encrypted.
SMTP	Submitting and relaying email.
POP3	Retrieving email, with retention behaviour set by the client.
IMAP	Accessing and synchronising server-held mailboxes.
MIME	Structures message content and attachments; not a transport protocol.

A BitTorrent example

BitTorrent divides a file into pieces that peers exchange. A swarm is the participating group. A seed has the complete content and can upload pieces. A downloading peer may upload too; the word “leecher” does not necessarily mean a malicious user. Trackers can help peers discover one another, while distributed discovery is also possible.

Security is part of the choice

A protocol name alone may not guarantee confidentiality or authenticity. Use protected versions or secure alternatives where appropriate. File-transfer tools such as SFTP use a different protocol from FTP. A networked application must also authenticate users and apply access controls at the service.

PAUSE & TRY

State the purpose of HTTP, FTP, POP3 and IMAP.

Chapter 4 · Question 4b · Adapted from GitHub exercises

Reserve a path or share capacity?

Circuit and packet switching

Circuit switching	Packet switching
Reserves communication capacity for a session.	Shares links by sending discrete packets.
Can provide predictable allocated capacity.	Uses capacity efficiently across variable traffic.
Reservation can be idle during pauses.	Queues, loss and variable delay need management.

Packets move hop by hop

A sender divides or packages data according to its protocols. Routers forward IP packets using destination information. Packets may follow different routes or arrive out of order, although this is not required for every transfer. Endpoint protocols handle the reliability or ordering their applications need.

Apply the comparison carefully

A reserved circuit still has propagation and processing delay; it does not mean zero latency or perfect reliability. Modern Internet video calls commonly use packet-switched networks, buffering, congestion control and other techniques. The source exercise about a preferred circuit assumes a scenario valuing reserved capacity; it is not a universal description of today's video conferencing.

PAUSE & TRY

List two benefits and one drawback of packet switching.

Chapter 4 · Question 5c · Adapted from GitHub exercises

Capacity is not responsiveness

Four problems to distinguish

Metric	What it tells you
Bandwidth / capacity	How much data a link can carry per second.
Throughput	The successful transfer rate actually achieved.
Latency	How long a transfer or response takes.
Jitter / loss	Variation in delay / missing packets.

Why a fast link can still feel slow

A wide road carries many cars, but a long journey still takes time. Similarly, high bandwidth does not remove propagation delay, queueing or server processing. Retransmitting lost data can add delay; interference and contention can reduce throughput. Measure the entire task, not just the advertised link rate.

Choose a response to the problem

Compress or summarise data when capacity is limited. Reduce unnecessary round trips when latency matters. Use timestamps to identify stale measurements. Buffering can smooth jitter for playback, but adds delay. Redundancy and reconnect logic can improve resilience, while local control keeps a time-critical action from depending on an unavailable remote link.

PAUSE & TRY

Explain resource optimisation when balancing local hardware processing against network transmission costs.

Chapter 6 · Question 5 · Adapted from GitHub exercises

Protect data and its meaning

Three different goals

Goal	Question
Security	Who can access, change or disrupt the system?
Privacy	How is personal data collected, used and shared?
Integrity	Has the data remained correct and unaltered?

Validation versus verification

Validation checks whether an input meets rules: range, type, length or a valid calendar date. It cannot prove a plausible value is true. Verification checks that data agrees with a source or expected copy, such as double entry or a transfer checksum. Checksums detect some errors; a cryptographic authentication mechanism is needed to resist deliberate modification by an attacker.

Use layers of protection

Strong authentication, least-privilege access, updates, backups and network filtering address different risks. Malware can damage or steal data; spyware monitors activity; phishing tricks a user; pharming redirects a user to an unintended destination. A firewall is not a substitute for secure software, and antivirus cannot promise to detect every threat.

PAUSE & TRY

Contrast validation, such as range and format checks, with verification, such as double entry and transfer checks, when protecting data quality.

Chapter 4 · Question 6b · Adapted from GitHub exercises

Keep messages confidential

From plaintext to ciphertext

Encryption uses an algorithm and a key to transform readable plaintext into ciphertext. Correctly implemented cryptography can protect confidentiality, but key management and authentication remain essential. Encryption alone does not prove who sent a message or ensure that a device receiving it is trustworthy.

Two approaches

Symmetric	Asymmetric
Uses a shared secret key.	Uses a related public/private key pair.
Efficient for bulk data encryption.	Supports mechanisms such as key establishment and signatures.
Both parties must obtain the secret securely.	The public key must be authenticated to its owner.

Combine their strengths

Secure protocols commonly use asymmetric mechanisms to authenticate or establish shared secrets, then symmetric authenticated encryption for application traffic. Keep private keys private. Do not invent an encryption scheme by substituting characters or treating a checksum as a secret; the design and implementation of cryptography require established protocols.

PAUSE & TRY

Compare symmetric and asymmetric cryptography in key management, typical performance and the handling of plaintext and ciphertext.

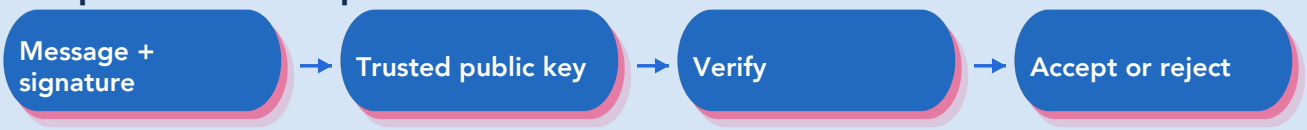
Chapter 4 · Question 7a · Adapted from GitHub exercises

Trust, signatures & certificates

A signature protects authenticity and integrity

A sender signs a message, usually through a hash-based construction, using a private signing key. A verifier checks the signature with the matching public key. A valid signature supports a claim about the key used and whether the signed content changed. It does not encrypt the message or prove that the message's claims are true.

A simplified verification path



Signing is not generally "encryption in reverse"; the operations depend on the signature algorithm.

Certificates and TLS

A digital certificate binds an identity, such as a domain name, to a public key and is signed by an issuer. A client checks the trust chain, validity and expected name. TLS uses authentication and cryptography to protect communication, including HTTPS. SSL is obsolete and should not be deployed. A certificate can authenticate a domain without proving the site is honest in every other respect.

PAUSE & TRY

What are SSL and TLS, and where is TLS used to secure network traffic today?

Chapter 4 · Question 7c · Adapted from GitHub exercises

Encryption modes & quantum ideas

Blocks and streams

A block cipher operates on fixed-size blocks; a mode specifies how it is used for longer messages. A stream cipher combines data with a generated keystream. Reusing a key/nonce combination incorrectly can undermine security. Authenticated encryption also detects unauthorised changes, rather than providing confidentiality alone.

Understand CBC as a historical example

Cipher block chaining combines a plaintext block with the previous ciphertext block before encryption, starting with an initialisation vector. Correct IV handling, padding and authentication matter. CBC is useful to understand, but a new application should use a vetted protocol and suitable authenticated-encryption construction rather than assembling its own cryptographic workflow.

What quantum key distribution does

QKD uses quantum signals and classical communication to establish key material. Its security analysis detects disturbance statistically under stated assumptions; it does not make every device or message unhackable. It requires specialised infrastructure and authenticated classical communication. Post-quantum cryptography is a different approach: algorithms designed to resist quantum attacks on conventional computers.

PAUSE & TRY

Compare symmetric and asymmetric cryptography in key management, typical performance and the handling of plaintext and ciphertext.

Chapter 4 · Question 7a · Adapted from GitHub exercises

Key terms · 04

Build your vocabulary

Term	Meaning
LAN / WAN / MAN / PAN	Local / wide-area / metropolitan / personal network scopes.
Client / server / peer	Service requester / provider / participant that can perform both roles.
Thin / thick client	Relies more on remote resources / performs substantial local work.
Switch / router	Forwards local frames / forwards IP packets between networks.
NIC / access point	A network interface / a wireless network access device.
Protocol / stack	Communication rules / a layered organisation of those rules.
Frame / IP packet	A link-layer unit / a routed Internet-layer unit.
URL / DNS	A resource locator / a distributed name-resolution system.
TCP / UDP	Ordered reliable byte-stream transport / datagram transport.

Use the terms in an explanation

Choose two terms and connect them in a sentence about a working system. A definition is useful when it helps you explain a mechanism, compare alternatives or justify a decision. Return to the relevant teaching page if you cannot yet give an example.

PAUSE & TRY

Differentiate between data security, data privacy and data integrity.

Chapter 4 · Question 6a · Adapted from GitHub exercises

Key terms · 04

Build your vocabulary

Term	Meaning
Bandwidth / throughput	Link capacity / achieved successful transfer rate.
Latency / jitter	Delay / variation in delay.
Security / privacy / integrity	Protection / appropriate personal-data use / accuracy and resistance to alteration.
Validation / verification	Checking rules / checking agreement with a source or expected copy.
Plaintext / ciphertext	Readable original data / encrypted data.
Symmetric / asymmetric	Shared-secret mechanisms / public–private key mechanisms.
Signature / certificate	An authenticity-and-integrity check / an issuer-signed identity-to-key binding.
TLS / SSL	Current secure-transport protocol family / obsolete predecessor.
QKD	Key establishment using quantum communication with specific assumptions and infrastructure.

Use the terms in an explanation

Choose two terms and connect them in a sentence about a working system. A definition is useful when it helps you explain a mechanism, compare alternatives or justify a decision. Return to the relevant teaching page if you cannot yet give an example.

PAUSE & TRY

Differentiate between data security, data privacy and data integrity.

Chapter 4 · Question 6a · Adapted from GitHub exercises

Practice · Chapter 4

Try first, then check feedback

Write your reasoning before opening the explained answers after the six chapter missions. Show units and intermediate steps for calculations. For comparisons, use the same criteria on both sides. Match the question number with the answer key in this complete edition.

Chapter 4 · Question 1c

Differentiate between a thick client and a thin client based on where processing and storage take place.

Chapter 4 · Question 5a

Explain what is meant by circuit switching.

Chapter 4 · Question 5b

Assume a video-conferencing scenario specifically requires reserved communication capacity. Explain why circuit switching could be preferred in that scenario. This is the source question's assumed context, not a claim that modern Internet video calls generally use circuits.

PAUSE & TRY

Differentiate between data security, data privacy and data integrity.

Chapter 4 · Question 6a · Adapted from GitHub exercises

Practice · Chapter 4

Try first, then check feedback

Write your reasoning before opening the explained answers after the six chapter missions. Show units and intermediate steps for calculations. For comparisons, use the same criteria on both sides. Match the question number with the answer key in this complete edition.

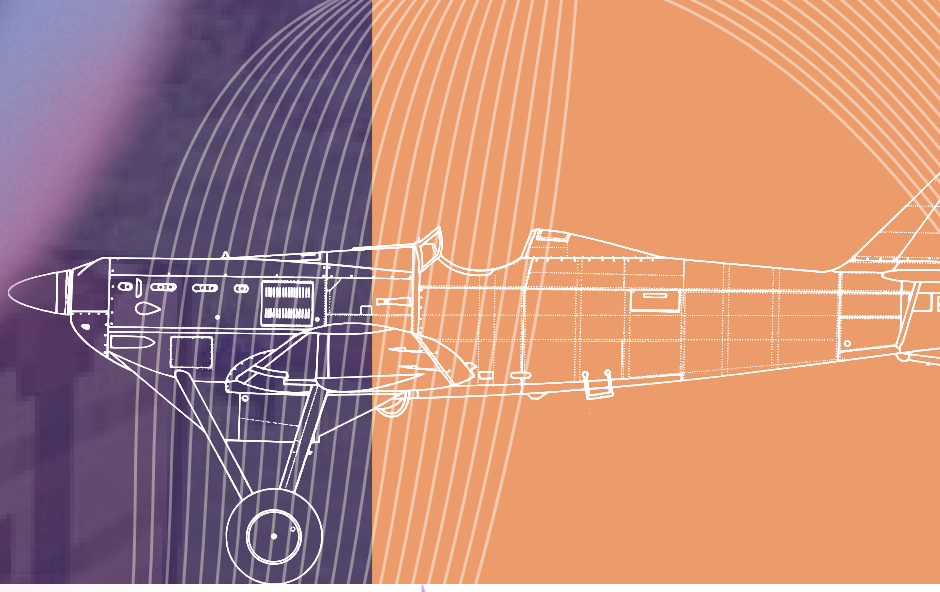
Chapter 4 · Question 6c

Name three mechanisms used to protect computer systems against threats.

PAUSE & TRY

Explain how a private signing key and matching public verification key support a digital signature that checks message origin and integrity. Distinguish signing from encryption.

Chapter 4 · Question 7b · Adapted from GitHub exercises



DUSK

APPLICATIONS

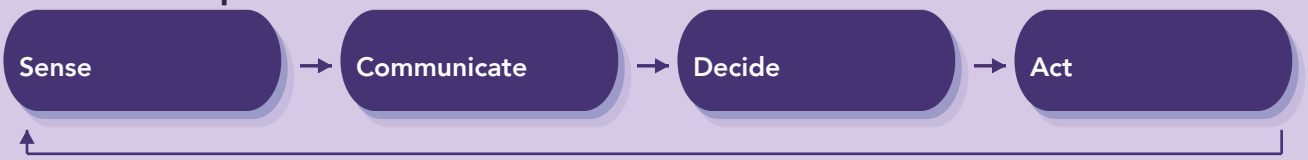
5

A system that senses and acts

Combine the parts

An integrated smart system connects physical hardware, communication and software to carry out a useful task. Sensors collect information; processing interprets it and selects an output; actuators change the world. A network lets components exchange data or use remote services. Some systems use fixed rules, some use learned models, and some combine both.

Follow one complete workflow



The changed environment supplies the next observation. The communication step may be local, remote or partly unnecessary for a self-contained controller.

Use the body analogy carefully

Sensors resemble senses, actuators resemble muscles and communication links resemble nerves. Software coordinates the response. This is an analogy for the roles, not a claim that the machine has a human mind. Explain the actual mechanism: what is measured, what message is sent, what computation runs and what physical output follows.

PAUSE & TRY

Define a smart system and explain how it integrates physical hardware, network connectivity and software into one workflow.

Chapter 5 · Question 1 · Adapted from GitHub exercises

Example: an irrigation system

Trace the flow of information

Stage	Illustrative design
Measure	Soil-moisture sensors report readings and timestamps.
Communicate	A low-power network sends readings to a local gateway.
Decide	A controller selects a watering action using targets and constraints.
Act	A driver energises a valve; later readings show the effect.

Add learning only when it has a role

A fixed threshold can open a valve when soil is dry. A learned policy might instead use moisture, recent rain and other state variables to trade off water use and crop needs. Reinforcement learning would require a defined reward and a process for updating the policy from experience. Merely connecting a sensor to Wi-Fi does not make the system learn.

Check assumptions before action

Sensors can drift; messages can arrive late; a valve can fail. Timestamp readings, detect implausible values and monitor the effect of each action. A real deployment needs domain-specific limits and testing. This example explains an architecture rather than prescribing watering durations for a particular crop.

PAUSE & TRY

Describe the role of actuators in translating a computed decision into a physical change.

Chapter 5 · Question 3b · Adapted from GitHub exercises

Real-time means meeting a deadline

Correctness includes timing

A real-time system must produce the required response within a specified deadline. “Real-time” does not simply mean very fast. Missing a hard deadline can make a system fail its requirements; missing a soft deadline reduces service quality. Batch work, such as a monthly report, usually has different timing needs.

Illustrative delay distance at 30 m/s

Delay before response	Distance travelled
500 ms = 0.500 s	30 × 0.500 = 15 metres
50 ms = 0.050 s	30 × 0.050 = 1.5 metres
5 ms = 0.005 s	30 × 0.005 = 0.15 metres

Interpret the calculation correctly

Distance during delay = speed × delay. This is only the distance travelled before a response begins, not total stopping distance. Sensing, processing, communication, actuation and physical braking all contribute to a real system. Moving a critical computation locally removes some communication delay, but it does not guarantee a particular safe outcome.

PAUSE & TRY

Define real-time processing and explain why communication delay and hardware performance matter for tasks such as collision avoidance.

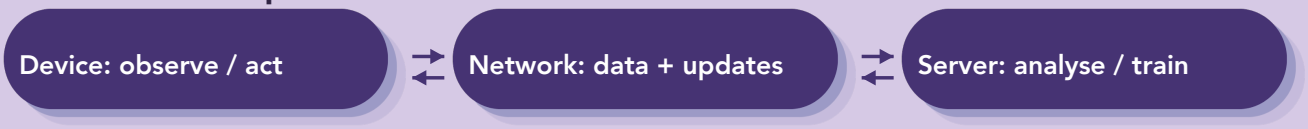
Chapter 5 · Question 2a · Adapted from GitHub exercises

Networking for a behaviour-based agent

Separate the fast loop from shared services

A useful architecture keeps time-critical sensing, policy evaluation and actuator control near the device. A network can carry telemetry, coordinate multiple devices or distribute reviewed model updates. Training may occur centrally while a fixed policy runs locally. A system can continue its local task during a link outage only if it was designed to do so.

Two connected responsibilities



Local action does not have to wait for every cloud exchange. This is an architectural choice, not a property of all AI systems.

Make messages interpretable

Include a device identity, timestamp, sequence number, units and model or policy version where needed. Reject stale or unauthorised commands. Delayed feedback must be associated with the action that produced it before a learning update is made. Network loss can reduce the evidence available for training; it should not silently become a reward of zero.

PAUSE & TRY

Explain resource optimisation when balancing local hardware processing against network transmission costs.

Chapter 6 · Question 5 · Adapted from GitHub exercises

Automation changes work and errors

A spectrum of responsibility

Mode	Who does what?
Manual	A human performs the task directly.
Assistance	The system supports selected actions or provides alerts.
Partial automation	The system handles parts of the task; the human supervises.
Higher autonomy	The system acts within a defined operating domain.

Benefits depend on the setting

Automation can improve repeatability, throughput and availability for suitable work. It can also shift errors from isolated human mistakes to systematic software, sensor or model failures affecting many operations. Do not assume that automation gives zero errors, unlimited uptime or a fixed percentage improvement.

Keep the operating domain explicit

A warehouse robot may work well on a mapped, maintained floor but fail on an unfamiliar surface. A human must understand what the system can handle, what it cannot handle and how a handover works. Formal driving-automation levels describe a specific domain; they should not be treated as a universal ladder for every smart appliance.

PAUSE & TRY

What is automation, and how can moving from manual operation to automated systems change efficiency and error patterns?

Chapter 5 · Question 2b · Adapted from GitHub exercises

Feedback can change a policy

Describe the learning mechanism

An RL agent collects transitions: a state, an action, a reward and a next state. A learning algorithm uses those transitions to update values or policy parameters. A positive reward does not mechanically guarantee that an action becomes more likely in every algorithm; updates depend on the method and its estimates of future return.

Design an illustrative delivery task

Element	Possible definition
State	Location, remaining energy and relevant obstacle information.
Action	Choose a permitted move.
Reward	Score progress and successful completion; penalise undesirable outcomes.
Constraint	Disallow unsafe actions even if a reward would favour them.

A deployment may stop learning

A trained policy can be evaluated repeatedly on new observations without changing its parameters. Online learning requires a controlled update process and evidence that changes improve performance. Separate “the action changed because the observation changed” from “the policy changed because the agent learned”.

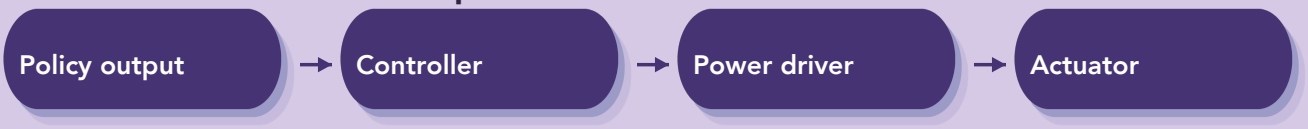
PAUSE & TRY

Explain how an RL-based system uses feedback or reward signals from its environment to update its behavioural policy or value estimates.

Chapter 5 · Question 3a · Adapted from GitHub exercises

Turn a decision into an action

A command needs an execution path



Feedback from position, current or other sensors can confirm whether the requested action happened.

Different physical outputs

Actuator	Example role
Electric motor / servo	Rotate a wheel or position a joint.
Solenoid valve	Open or close a fluid path.
Hydraulic actuator	Produce force using pressurised liquid.
Pneumatic actuator	Produce movement using compressed air.

Distinguish information from energy

A software value such as “target angle = 15 degrees” is information. Interface electronics convert it into a suitable control signal; a powered mechanism supplies the force. Some systems use analogue voltages, others PWM or digital commands. A DAC alone normally cannot supply motor power. Mechanical limits, response time and feedback all affect whether an intended action is achieved.

PAUSE & TRY

Describe the role of actuators in translating a computed decision into a physical change.

Chapter 5 · Question 3b · Adapted from GitHub exercises

Help people understand the system

Human-AI interaction is part of HCI

Human-computer interaction studies how people use computing systems. Human-AI interaction focuses on challenges raised by AI, including uncertainty, adaptation, delegation and trust. It is not a strict division between deterministic old interfaces and always-probabilistic new ones. The important question is what the person needs to understand and control.

Useful interface information

Show	Why it helps
Current mode and operating limits	The user knows what the system is doing.
Relevant evidence and uncertainty	The user can judge an output appropriately.
Planned action and reason	The next behaviour is less surprising.
Override, handover and recovery	The user can respond when the system cannot continue.

Avoid over-trust and under-trust

Unclear automation can cause misuse through excessive reliance, or disuse when useful assistance is rejected. A handover should communicate the state of the task and why control is changing. A confident-looking score is not enough: the interface should support an informed decision and an appropriate human response.

PAUSE & TRY

Identify a usability or safety challenge that can occur when a smart system's autonomous behaviour lacks transparency.

Chapter 5 · Question 4b · Adapted from GitHub exercises

Evaluate more than performance

Three important ethical concerns

Concern	Design question
Fairness	Who benefits, who is disadvantaged, and how was this measured?
Transparency	Can affected people understand the system's role and limitations?
Accountability	Who is responsible for deployment, oversight and correction?

Reward design can encode priorities

A reward that values only throughput can encourage neglect of other goals. Historical data can contain unequal representation or biased decisions. Evaluate outcomes across relevant groups and situations rather than assuming an algorithm is neutral because it uses numbers. The appropriate criteria depend on the task and the people affected.

Make responsibility practical

Document the intended use, data sources, limitations and monitoring process. Provide channels to question or correct consequential outcomes. Ethical and legal requirements vary by context and jurisdiction; no single checklist proves a system is responsible. For high-stakes uses, performance testing and meaningful human oversight are essential parts of the deployment decision.

PAUSE & TRY

Define ethical AI and discuss the challenges of maintaining transparency, fairness and accountability in smart systems.

Chapter 5 · Question 5a · Adapted from GitHub exercises

Opportunities bring responsibilities

Illustrative connected applications

Application	Opportunity	Risk to consider
Traffic signals	Respond to changing demand.	Unsafe control or unequal service.
Water monitoring	Detect leaks and reduce waste.	Faulty sensors or unauthorised commands.
Street lighting	Adjust lighting to local need.	Privacy, coverage and maintenance.
Connected health monitoring	Support timely review of measurements.	Sensitive data and incorrect alerts.

Measure outcomes, not just activity

More sensor readings or more automated actions do not necessarily mean better service. Compare meaningful outcomes such as resource use, reliability, accessibility and user experience. Any claimed improvement needs evidence from the particular deployment; percentages from an unspecified example are not general guarantees.

Every connected device adds responsibilities

Connected cameras, locks, sensors and controllers need updates, access management and appropriate data protection. Minimise unnecessary personal data, limit retention and protect communication. Consider what happens if a device is compromised or the network is unavailable. Convenience should be assessed alongside maintainability, privacy and safe operation.

PAUSE & TRY

Identify one opportunity and one security or privacy risk associated with connecting everyday physical devices to networks.

Chapter 5 · Question 5b · Adapted from GitHub exercises

Key terms · 05

Build your vocabulary

Term	Meaning
Smart system	Hardware, communication and software integrated for a useful task.
Real-time processing	Processing that must satisfy a specified timing deadline.
Automation	Technology performing some or all parts of a task with reduced direct human operation.
Edge / cloud computing	Processing near the source / processing on remote shared infrastructure.
Human-AI interaction	Interaction challenges involving AI, including uncertainty, delegation and trust.
Ethical AI	Design and use that consider fairness, accountability, transparency and impacts on people.
Operating domain	The conditions under which a system is intended to function.
Telemetry	Measurements and status information communicated for monitoring or analysis.

Use the terms in an explanation

Choose two terms and connect them in a sentence about a working system. A definition is useful when it helps you explain a mechanism, compare alternatives or justify a decision. Return to the relevant teaching page if you cannot yet give an example.

PAUSE & TRY

Define a smart system and explain how it integrates physical hardware, network connectivity and software into one workflow.

Chapter 5 · Question 1 · Adapted from GitHub exercises

Practice · Chapter 5

Try first, then check feedback

Write your reasoning before opening the explained answers after the six chapter missions. Show units and intermediate steps for calculations. For comparisons, use the same criteria on both sides. Match the question number with the answer key in this complete edition.

PAUSE & TRY

Define human-AI interaction and explain the additional challenges it introduces within human-computer interaction.

Chapter 5 · Question 4a · Adapted from GitHub exercises

MIDNIGHT

STRENGTH & LIMITATIONS

6

Judge the whole system

Strengths and limitations interact

Part	Strength	Limitation
Hardware	Senses, computes and acts locally.	Power, heat, memory and physical reliability.
Networks	Share resources and coordinate devices.	Delay, loss, availability and security.
AI / control software	Selects actions and may learn from experience.	Model errors, limited evidence and unclear objectives.

There is no universally best architecture

Choose a design for its task, operating conditions and consequences of failure. A model with excellent offline accuracy may be unsuitable if it misses a deadline or requires too much memory. A highly connected system may be fragile if local operation depends on one remote service. A simple fixed-rule controller can be appropriate for a well-defined task.

Make the comparison explicit

State the intended outcome, identify constraints, compare options and define measures for success. Evaluate the combination of hardware, networking, algorithms and human operation. A component's specification is useful evidence, but it is not proof that the complete application meets its requirements.

PAUSE & TRY

Explain resource optimisation when balancing local hardware processing against network transmission costs.

Chapter 6 · Question 5 · Adapted from GitHub exercises

Four physical budgets

Every device has limits

Power | Heat | Memory | Processing

A design must fit all four budgets, not just the model's arithmetic requirement.

Understand the constraint

Budget	What can go wrong
Power	A battery drains too quickly for the required mission.
Heat	Thermal throttling reduces sustained performance.
Memory	Weights, inputs or intermediate results do not fit.
Processing	An inference or control computation misses its deadline.

Optimise with evidence

A smaller model, quantised representation, reduced input resolution or lower update rate can reduce cost, but may change accuracy or responsiveness. Measure the resulting behaviour. Dedicated accelerators can improve efficiency on supported workloads, but data conversion, memory movement and idle power remain relevant. Local computing removes a remote round trip; it does not mean zero total latency.

PAUSE & TRY

Explain how thermal limits, power consumption, memory capacity and processing speed constrain complex AI models on small embedded devices.

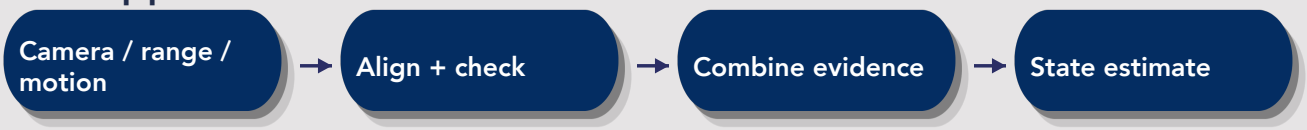
Chapter 6 · Question 1 · Adapted from GitHub exercises

Reliable decisions need reliable inputs

Sensors are imperfect

Noise creates random variation; drift changes a sensor's behaviour over time. Obstruction, calibration error or environmental conditions can produce misleading observations. Sensor fusion combines information from multiple sources, often after time alignment and filtering. Different sensors can have complementary strengths, but they can also share common failure causes.

A fusion pipeline



Do not combine incompatible units or measurements from different times as if they described the same instant.

Define reliability under conditions

Reliability concerns performing an intended function for a stated period under stated conditions. MTBF is a statistical measure for repairable systems, not a promise that a particular unit cannot fail earlier. Monitor drift, error rates, temperature and data quality. Redundancy can help when failures are sufficiently independent and the system can recognise and manage disagreement.

PAUSE & TRY

Explain how thermal limits, power consumption, memory capacity and processing speed constrain complex AI models on small embedded devices.

Chapter 6 · Question 1 · Adapted from GitHub exercises

Grow without losing coordination

Three network qualities

Quality	Meaning
Scalability	Handle more devices or work at acceptable performance.
Interoperability	Exchange data and use it correctly across different systems.
Resilience	Maintain or recover an acceptable service during disruption.

Capacity is only one part of scaling

Adding hundreds of IoT devices can increase addressing, authentication, message traffic, storage and maintenance work. Agree on data formats, units, timestamps and interface versions as well as the physical link. Two products using Wi-Fi do not automatically interpret one another's messages correctly.

Plan for failures

Redundant links, retry policies, local buffering and controlled reconnection can improve resilience. A mesh can route around some failures when an alternative path exists and the protocols support it. It does not guarantee communication through every outage. Test overload and partial failure, not just the normal case with every device online.

PAUSE & TRY

Define scalability and interoperability. Explain why both matter when adding hundreds of IoT devices from different manufacturers.

Chapter 6 · Question 2 · Adapted from GitHub exercises

Put each computation where it fits

Three deployment patterns

Location	Benefit	Cost or limit
On-device edge	Low communication delay; local data.	Limited local resources and maintenance.
Nearby gateway / fog	Aggregates local devices.	Another service to size and manage.
Remote cloud	Shared compute and flexible capacity.	Network dependency, latency and transfer cost.

An illustrative warehouse design

Each robot handles immediate motion control locally. A shared service allocates tasks and coordinates inventory. Robots send selected telemetry and receive authorised updates. If the coordination service is unavailable, the design defines whether a robot pauses, completes a limited task or enters another safe state; continued operation must not be assumed.

Optimise across the whole path

Compare local compute energy with transmission energy, available bandwidth, deadlines, privacy and model capacity. Summarising data locally can reduce transfer volume. Centralised analysis can reveal patterns across devices. The useful decision is which part of the workload belongs where, not whether edge or cloud is always superior.

PAUSE & TRY

Explain resource optimisation when balancing local hardware processing against network transmission costs.

Chapter 6 · Question 5 · Adapted from GitHub exercises

Predict an outcome or model behaviour?

Two useful questions

Predictive analytics	Behavioural modelling
What is likely to happen?	How does an agent or system behave?
Forecasts, scores or probability estimates.	Rules, transitions, trajectories or action patterns.
Example: estimate a component's failure risk.	Example: model a robot's response to an obstacle.

The approaches can overlap

A behavioural model can be used to make predictions, and predictive features can inform an action policy. They are not mutually exclusive categories defined by one kind of data. Choose the model and evaluation method based on the question being asked and the evidence available.

Adaptation is not guaranteed generalisation

Adaptive intelligence describes changing behaviour appropriately using new information or experience. Autonomy describes operating with limited direct human control. An autonomous device may use a fixed policy; an adaptive model may still need supervision. Performance in familiar conditions does not prove reliability in a new environment.

PAUSE & TRY

Compare predictive analytics and behavioural modelling in how they use evidence to inform future decisions or actions.

Chapter 6 · Question 4 · Adapted from GitHub exercises

Understand why a policy acted

Explainability is a practical capability

Explainability concerns making relevant aspects of a model's behaviour understandable to people. A small rule set or Q table may be inspectable. A deep RL policy can be harder to interpret because many parameters contribute to each output. Complexity does not mean analysis is impossible, but a simple story about one decision may be incomplete or misleading.

Look beyond one successful outcome

Inspect representative cases, counterexamples, changed conditions and failure logs. Separate a faithful explanation of the computation from a plausible narrative generated after the fact. An explanation does not establish that the action was correct; validation requires evidence about outcomes and requirements.

Reward errors and bias

An agent rewarded only for speed may neglect an important constraint. Training environments can under-represent some conditions or groups. Distribution shift can reduce performance when deployment differs from training. Track both overall performance and relevant subsets. Correcting a reward or dataset requires re-evaluation of the resulting policy.

PAUSE & TRY

Define explainability and explain why some deep reinforcement-learning models are difficult to interpret.

Chapter 6 · Question 6 · Adapted from GitHub exercises

Balance benefit, risk & oversight

Questions for a deployment review

Area	Evidence to seek
Security	Authenticated updates, restricted access and recovery plans.
Privacy	Necessary data collection, appropriate retention and access.
Fairness	Evaluation across relevant people and conditions.
Accountability	Named responsibilities, review and correction routes.
Reliability	Measured deadlines, failure handling and monitoring.

Physical effects raise the stakes

A connected controller can change the physical world, so a corrupted measurement or unauthorised command may have consequences beyond a damaged file. Protect sensor inputs, model updates and actuator commands according to the task. Test how the system responds to missing, stale and contradictory data.

Finish with a justified judgement

State the benefit you expect, the evidence supporting it, the conditions under which it applies and the remaining limitations. Decide what a human must monitor or approve and how the system should respond when its assumptions fail. The strongest design is the one suited to its purpose, with limitations that are understood and managed.

PAUSE & TRY

Identify two security concerns and two ethical implications when a network of smart devices makes high-stakes decisions with limited human oversight.

Chapter 6 · Question 7 · Adapted from GitHub exercises

Key terms · 06

Build your vocabulary

Term	Meaning
Sensor integration	Connecting and coordinating measurements for a computing system.
Hardware constraints	Limits such as power, thermal capacity, memory and compute time.
Reliability	Performance of the intended function over time under stated conditions.
Sensor drift	A gradual change in measurement behaviour or calibration.
Scalability	Ability to handle growth at acceptable performance.
Interoperability	Ability to exchange information and use it correctly across systems.
Resilience	Ability to maintain or recover acceptable service through disruption.
Resource optimisation	Balancing computation, communication, energy and other costs for a goal.
Adaptive intelligence	Changing behaviour appropriately using new information or experience.

Use the terms in an explanation

Choose two terms and connect them in a sentence about a working system. A definition is useful when it helps you explain a mechanism, compare alternatives or justify a decision. Return to the relevant teaching page if you cannot yet give an example.

PAUSE & TRY

Define adaptive intelligence and explain how it can help an agent respond to changing conditions, without assuming that adaptation guarantees success.

Chapter 6 · Question 3 · Adapted from GitHub exercises

Key terms · 06

Build your vocabulary

Term	Meaning
Autonomy	Operating with limited direct human intervention.
Predictive analytics	Using evidence to forecast outcomes.
Behavioural modelling	Representing or analysing behaviour and interactions.
Explainability	Making relevant aspects of model behaviour understandable.
Bias / distribution shift	Systematic distortion / a difference between training and deployment conditions.

Use the terms in an explanation

Choose two terms and connect them in a sentence about a working system. A definition is useful when it helps you explain a mechanism, compare alternatives or justify a decision. Return to the relevant teaching page if you cannot yet give an example.

PAUSE & TRY

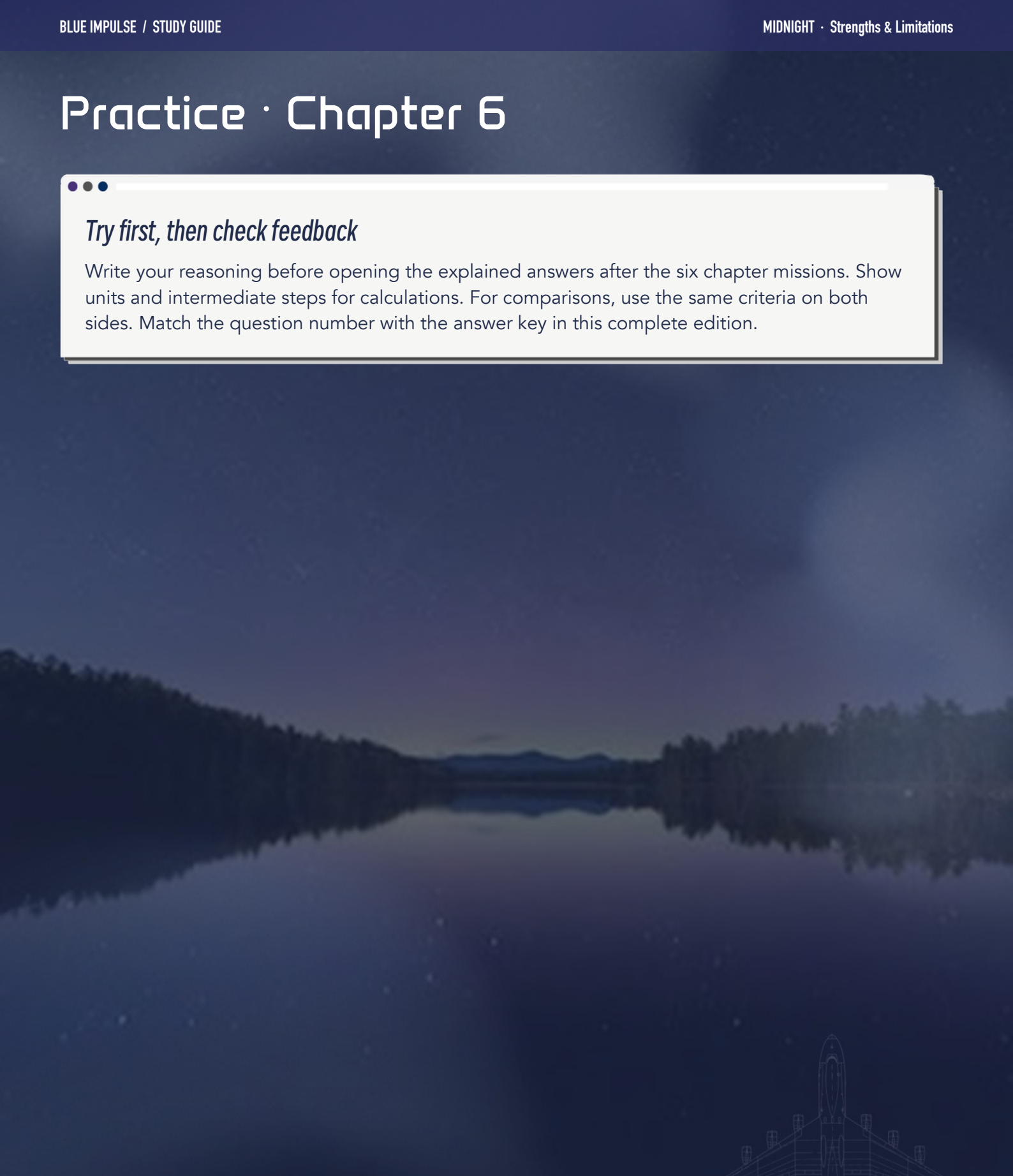
Define adaptive intelligence and explain how it can help an agent respond to changing conditions, without assuming that adaptation guarantees success.

Chapter 6 · Question 3 · Adapted from GitHub exercises

Practice · Chapter 6

Try first, then check feedback

Write your reasoning before opening the explained answers after the six chapter missions. Show units and intermediate steps for calculations. For comparisons, use the same criteria on both sides. Match the question number with the answer key in this complete edition.



PAUSE & TRY

Define adaptive intelligence and explain how it can help an agent respond to changing conditions, without assuming that adaptation guarantees success.

Chapter 6 · Question 3 · Adapted from GitHub exercises

Sources & acknowledgements

Learning materials

This guide is an edited synthesis of the six chapter PDFs supplied by Blue Impulse, with learning outcomes, key terms and exercises from the project repository at revision 9a645d0. The team's source notes credit Cambridge International AS & A Level Computer Science and disclose use of Gemini, Claude and ChatGPT. The supplied files do not identify a complete textbook edition or bibliography.

Artwork and production

The unchanged cover and six labelled dividers come from study guide art.pdf. Its window boxes, folder panels, diagram shapes, aircraft and sky images are reused throughout; the supplementary art PDF guides the layouts. Anta, DIN Condensed, Smiley Sans and Avenir follow the written font brief. Neogpt assisted with editing, diagrams, layout and PDF production.

How to read this edition

Page numbers refer to physical PDF pages, including the cover and dividers. Each teaching or reference page ends with a question from the team's exercise files. The complete Stage 2 edition includes six missions, resources and explained answers after the core chapters. Wording is lightly edited for clarity and technical accuracy. Worked examples teach methods using separate examples. The original art's printed chapter labels determine the divider order.

PAUSE & TRY

Identify two security concerns and two ethical implications when a network of smart devices makes high-stakes decisions with limited human oversight.

Chapter 6 · Question 7 · Adapted from GitHub exercises

Active learning route and feedback

Blue Impulse Study Guide V1.0 — Stage 2 complete edition, 7 November 2026

This complete edition preserves the original cover, chapter dividers and developed teaching pages, then adds the active learning missions, optional resource introductions and explained exercise answers. The story follows Pip, a fictional campus delivery rover; robot-7 is its identifier in the packet exercise. Fictional model measurements are labelled and are not research results.

How to use the complete guide

Start with the learning goals and core explanations. In every chapter, pause at the exercise and make a prediction before reading feedback. Use the matching chapter mission below to apply the ideas, record the result and explain why it happens. Then compare the expected answer and try a changed input. The answer key is placed after all six missions so learners can attempt the questions first.

Prerequisites are basic arithmetic and the earlier chapters in sequence. Paper and a calculator are sufficient. Optional code is supplied in the submission's supporting folder and needs Python 3.9 or newer, with no extra packages or account. The separate editable Test Bank provides 54 balanced questions for review; its questions come before its answer key. Human peer review remains pending, as documented in the package's review record.

Complete edition navigation

The original six-chapter contents and bookmarks remain usable. Additional PDF bookmarks open each mission, resource introduction and explained-answer section. The teaching topics remain named consistently across Markdown, slides and videos; revised videos cite chapter number, chapter name and subtopic rather than page numbers.

Mission 1 — The parcel total is wrong

ACT-01 — The parcel total is wrong

Chapter 1 — The Fundamentals

Pip's delivery receipt says 200 parcels. The team expected 7. A programmer has confused an address with a value. Can you find the mistake?

Goal: Trace stored instructions and distinguish an immediate value, an address and the content stored there.

Preparation: Paper and pencil are sufficient. Optional: Python 3.9 or newer and supporting/BlueImpulse_Demonstrations.py. This is an invented teaching ISA, not a real processor.

Task:

1. Write columns for PC before fetch, instruction, ACC after execution and memory[200]. Start at PC 100, ACC 0 and memory[200] 0.

2. Run: 100 LOAD #3; 101 ADD #4; 102 STORE 200; 103 HALT. The # symbol means a literal value. PC advances by one when each instruction is fetched.

3. Predict the final result before checking the output. Then replace ADD #4 with ADD #9 and trace again.

4. Optional command: python3 supporting/BlueImpulse_Demonstrations.py --topic execution --add 9. Compare its state trace with your table.

Observe and record: Record ACC after every instruction, memory[200] after STORE, and final PC. Note the difference between a number used as data and a number used as an address.

Reflect: Why does STORE 200 copy 7 rather than replace the accumulator with 200? Which parts of the model would depend on a real ISA?

Feedback — check after trying: First run: after LOAD, ACC 3; after ADD, ACC 7; after STORE, memory[200] 7; after HALT, final PC 104. The modified run stores 12. STORE's operand is a destination address. PC increments, instruction size and register names are model choices, not universal CPU rules.

Optional resource: R2 (full introduction and offline alternative follow).

R2 — Nand to Tetris — Project 5: Computer Architecture

Noam Nisan and Shimon Schocken / Nand to Tetris.

https://www.nand2tetris.org/project05

Relevant section: Project overview, proposed computer architecture and hardware simulator guidance

Why use it: A concrete educational computer architecture connects logic, CPU state and memory.

Learner task: After ACT-01, compare the purpose of an instruction address and a data-memory address with the project's computer model. Name one part the models share and one detail that differs.

Limits: The Hack computer is a different ISA; its instructions and registers must not be substituted directly into this guide's toy program. Software installation is optional and not required for the paper task.

Offline alternative: ACT-01 includes its full program, trace and expected output. ACT-03 provides a complete paper pipeline grid specification.

Access: Optional public web resource; no account required to read the cited page. Signed-out HTTP access was checked on 7 November 2026. Availability on every campus network is not guaranteed. All essential task instructions and feedback are supplied locally.

Mission 2 — One lucky route is not a policy

ACT-02 — One lucky route is not a policy

Chapter 2 — Behaviorism & Behaviorist AI

Pip takes a new route and arrives early once. "Best route forever!" it announces. Its teammate asks whether one reward is enough evidence.

Goal: Calculate a Q-learning update and distinguish an estimate from a guaranteed outcome.

Preparation: Calculator or paper. Optional standard-library Python script. Know reward, policy, value, alpha and gamma from this chapter.

Task:

1. Use old Q = 2, alpha = 0.5, reward = 1, gamma = 0.9 and best next Q = 4. Predict whether the estimate will rise or fall.

2. Calculate target = reward + gamma × best next Q; error = target − old Q; change = alpha × error; new Q = old Q + change.

3. Repeat with the same old Q, alpha and reward but a terminal next state. Set future value to zero.

4. Optional command: python3 supporting/BlueImpulse_Demonstrations.py --topic rl. Compare values. Explain whether either update proves a route is safest.

Observe and record: Record the target, temporal-difference error, change and new value for both cases.

Reflect: Why is the future term zero at termination? Why can exploration remain useful after a positive outcome? What safety rule would constrain a real robot's exploration?

Feedback — check after trying: Non-terminal: target 4.6, error 2.6, change 1.3, new Q 3.3. Terminal: target 1, error −1, change −0.5, new Q 1.5. These are updated estimates, not proofs. Use many relevant transitions and evaluate behaviour against the task, including safety constraints.

Optional resource: R1 (full introduction and offline alternative follow).

R1 — Reinforcement Learning: An Introduction, second edition

Richard S. Sutton and Andrew G. Barto, MIT Press, 2018.

http://incompleteideas.net/book/the-book-2nd.html

Relevant section: Chapter 3 (finite Markov decision processes) and Section 6.5 (Q-learning)

Why use it: Primary textbook support for the agent/environment distinction, expected return and the Q-learning update.

Learner task: After ACT-02, explain in your own words what the bootstrapped target includes, then compare it with the terminal-state case.

Limits: The book is mathematically deeper than this beginner guide. Start with the supplied worked calculation; do not treat the browser page as required activity content.

Offline alternative: The formula, both complete calculations and interpretation are included in ACT-02 and the chapter, so the task works offline.

Access: Optional public web resource; no account required to read the cited page. Signed-out HTTP access was checked on 7 November 2026. Availability on every campus network is not guaranteed. All essential task instructions and feedback are supplied locally.

Mission 3 — The kitchen with four stations

ACT-03 — The kitchen with four stations

Chapter 3 — Hardware

Pip needs three sensor calculations. One worker finishes all four steps of a job before starting the next. Another team proposes four stations. Will each job suddenly take one step?

Goal: Distinguish pipeline throughput from the latency of one instruction and identify the effect of a dependency.

Preparation: Draw a grid with cycles 1–8 as columns and instructions I1–I3 as rows. Use stages F (fetch), D (decode), E (execute), W (write back). Assume one cycle per stage, independent instructions and no stalls initially.

Task:

1. Schedule I1 in F/D/E/W across cycles 1–4; start I2 at cycle 2 and I3 at cycle 3.

2. Predict the finish cycles and compare total elapsed cycles with three instructions run without overlap.

3. Now assume I2 needs a result from I1 that is unavailable until the end of I1's W. State this dependency explicitly; postpone I2's E to cycle 5 and propagate later stages without letting instructions overtake.

4. Optional command: python3 supporting/BlueImpulse_Demonstrations.py --topic hardware. Use its ideal trace for the first case. Treat your stalled case as a separate scheduling model.

Observe and record: Ideal finish cycles are 4, 5 and 6. Mark every occupied stage and every inserted wait. Do not equate elapsed cycles with the sum of all stage work.

Reflect: What improved: single-instruction latency or completed instructions per unit time? Why must a claim of speedup list its assumptions?

Feedback — check after trying: Ideal case takes six cycles rather than twelve sequential stage cycles, but each instruction still uses four stages. In the stated dependency model I2 executes at 5 and writes at 6; I3 must wait behind it and can execute at 6 and write at 7. Real forwarding, stage timings and hazards differ, so this trace does not predict every CPU.

Optional resource: R2 (full introduction and offline alternative follow).

R2 — Nand to Tetris — Project 5: Computer Architecture

Noam Nisan and Shimon Schocken / Nand to Tetris.

https://www.nand2tetris.org/project05

Relevant section: Project overview, proposed computer architecture and hardware simulator guidance

Why use it: A concrete educational computer architecture connects logic, CPU state and memory.

Learner task: After ACT-01, compare the purpose of an instruction address and a data-memory address with the project's computer model. Name one part the models share and one detail that differs.

Limits: The Hack computer is a different ISA; its instructions and registers must not be substituted directly into this guide's toy program. Software installation is optional and not required for the paper task.

Offline alternative: ACT-01 includes its full program, trace and expected output. ACT-03 provides a complete paper pipeline grid specification.

Access: Optional public web resource; no account required to read the cited page. Signed-out HTTP access was checked on 7 November 2026. Availability on every campus network is not guaranteed. All essential task instructions and feedback are supplied locally.

Mission 4 — The fast network sends a late warning

ACT-04 — The fast network sends a late warning

Chapter 4 — Networking

Pip's dashboard celebrates a high link rate. A stop warning arrives half a second late. "But the Wi-Fi is fast!" says the operator. The parcels keep moving.

Goal: Separate link capacity from response latency and calculate distance travelled during communication delay.

Preparation: Paper or calculator. Use a constant illustrative speed of 30 m/s and delays 500, 50, 5 and 200 milliseconds. This is not a campus-rover speed claim or a braking simulation.

Task:

1. Predict which delay causes the greatest extra travel before a response.

2. Convert each delay to seconds by dividing by 1,000. Calculate delay distance = speed × delay.

3. Create a two-column table of delay and distance. Explain why increasing Mb/s alone need not remove propagation, queueing or processing delay.

4. Optional command: python3 supporting/BlueImpulse_Demonstrations.py --topic network. Compare results and change the speed in your own calculation to 2 m/s.

Observe and record: Record units at each step and compare ratios: a ten-times smaller delay creates ten-times smaller delay distance at constant speed.

Reflect: What does the calculation omit? How could jitter or occasional long queueing delays matter even when average latency is low?

Feedback — check after trying: 500/50/5/200 ms correspond to 0.5/0.05/0.005/0.2 s and 15/1.5/0.15/6 m at 30 m/s. At 2 m/s they give 1/0.1/0.01/0.4 m. This excludes reaction after receipt, actuator lag and braking distance; do not call it total stopping distance.

Optional resource: R3 (full introduction and offline alternative follow).

R3 — RFC 1122 — Requirements for Internet Hosts: Communication Layers

IETF / R. Braden, editor, 1989.

https://www.rfc-editor.org/rfc/rfc1122

Relevant section: Section 1.1.3 (Internet protocol suite) and the link, IP and transport sections

Why use it: An authoritative foundation for the layered Internet model used in Chapter 4.

Learner task: Draw the four teaching layers and place HTTP, TCP, IP and Ethernet/Wi-Fi by role. Explain why a large link capacity does not set an end-to-end response time.

Limits: This is a technical historical standards document; current protocols have later updates. Use it for layer roles, not as a complete modern security checklist.

Offline alternative: The chapter and ACT-04 contain the layer explanations and all delay inputs/results offline.

Access: Optional public web resource; no account required to read the cited page. Signed-out HTTP access was checked on 7 November 2026. Availability on every campus network is not guaranteed. All essential task instructions and feedback are supplied locally.

Mission 5 — Yesterday's command arrives today

ACT-05 — Yesterday's command arrives today

Chapter 5 — Applications

Pip has stopped safely. A delayed "move" packet arrives and looks correctly formatted. Should it obey a message just because the bytes make sense?

Goal: Apply identity, unit, version, freshness and sequence checks while distinguishing secure transport from safe command handling.

Preparation: Receiver device ID robot-7; expected units m; policy_version 3; maximum age 200 ms; last accepted sequence 41. Assume the upstream authentication check is true for these examples. The toy Boolean is not a cryptographic implementation.

Task:

1. Inspect packets in order: A = sequence 42, age 50 ms; B = sequence 43, age 600 ms; C = sequence 42, age 60 ms. All other listed fields match.

2. For each packet, decide ACCEPT, REJECT STALE or REJECT DUPLICATE. Only update last accepted sequence when a packet is accepted.

3. Predict what would happen if a developer updated last sequence before checking age, and explain why reject paths should not corrupt accepted-state bookkeeping.

4. Optional command: python3 supporting/BlueImpulse_Demonstrations.py --topic networked-ai. Test the same order and compare receiver state.

Observe and record: Write the decision and last accepted sequence after each packet. Identify which check fails; do not invent a new packet timestamp.

Reflect: What must a real deployment add for clock assumptions, authentication, session resets and a safe local response when the network fails?

Feedback — check after trying: A is ACCEPT and advances last sequence to 42. B is STALE; last sequence stays 42. C is DUPLICATE; last sequence stays 42. Version, identity and unit checks add semantic protection. Real systems need authenticated messages, replay/session design, trustworthy timing and explicit fallback, not only this toy validation function.

Optional resource: R4 (full introduction and offline alternative follow).

R4 — RFC 8446 — The Transport Layer Security (TLS) Protocol Version 1.3

IETF / Eric Rescorla, 2018.

https://www.rfc-editor.org/rfc/rfc8446

Relevant section: Section 1 (Introduction) and Section 8 (0-RTT and anti-replay)

Why use it: Distinguishes transport confidentiality/integrity from application-level decisions, including replay considerations.

Learner task: After ACT-05, explain why authenticated transport alone does not decide whether an old command is still appropriate to execute.

Limits: The Python receiver is a validation demonstration, not a TLS implementation. Do not implement cryptography from this worksheet.

Offline alternative: ACT-05 includes the packet fields, state transitions and rejection explanations; no network service or account is needed.

Access: Optional public web resource; no account required to read the cited page. Signed-out HTTP access was checked on 7 November 2026. Availability on every campus network is not guaranteed. All essential task instructions and feedback are supplied locally.

Mission 6 — Choose a system, not a leaderboard

ACT-06 — Choose a system, not a leaderboard

Chapter 6 — Strengths & Limitations

A larger model wins an accuracy contest but overheats Pip's computer. A cloud model is accurate too, until the network drops. The team must choose a deployment, not a single impressive number.

Goal: Evaluate edge/cloud placement against explicit latency, memory, power, privacy and resilience constraints.

Preparation: Use this fictional design table: local small model = 30 ms compute, 180 MB working memory, 4 W, 92% measured task accuracy; local large model = 90 ms, 700 MB, 9 W, 96%; remote model = 20 ms server compute plus 100–300 ms round trip, 98%, uploads images. Device budgets: 512 MB, 6 W and 80 ms deadline. Treat accuracies as supplied hypothetical measurements, not research results.

Task:

1. Eliminate designs that violate hard local memory, power or timing budgets. For the remote model add the network and server times.

2. Choose where emergency control and overnight fleet analytics should run. State how the system behaves during lost connectivity.

3. A software update improves the small model to 94% but needs 70 ms and 5 W. Decide what to test before accepting it; do not assume that one benchmark settles safety.

4. Draw a two-column decision record: constraint/evidence, then choice/trade-off. Propose one privacy safeguard and one monitoring measure.

Observe and record: Separate hard failures from preferences and record the worst stated time rather than only the mean. Identify unknowns such as warm-device timing and test-data coverage.

Reflect: Why is 98% accuracy not sufficient to choose the remote model for the 80 ms task? Which human decision remains after the calculations?

Feedback — check after trying: The small local model fits the stated budgets. The large local model violates memory, power and latency. The remote option takes 120–320 ms and misses the deadline; it may suit slower analytics with privacy controls. The updated small model fits the listed limits but leaves only 10 ms timing margin, requiring end-to-end/warm-device testing, relevant accuracy evaluation and a fallback. Valid alternatives earn credit if their assumptions and trade-offs are explicit.

Optional resource: R5 (full introduction and offline alternative follow).

R5 — Artificial Intelligence Risk Management Framework (AI RMF 1.0)

NIST, NIST AI 100-1, January 2023.

https://www.nist.gov/itl/ai-risk-management-framework

Relevant section: Framework functions Govern, Map, Measure and Manage; trustworthy AI characteristics

Why use it: Connects measured performance with context, impacts, monitoring and responsibility.

Learner task: For ACT-06, assign one concrete action to each framework function: identify an owner, describe the operating domain, measure a relevant failure and plan a response.

Limits: A voluntary risk-management framework is not certification that a product is safe, fair or legally compliant. Context and evidence still matter.

Offline alternative: ACT-06 supplies the design table and expected trade-off reasoning. A suitable four-action response is: Govern — name an accountable operator; Map — define the campus operating domain; Measure — test missed deadlines and uneven accuracy; Manage — maintain fallback and an incident response process.

Access: Optional public web resource; no account required to read the cited page. Signed-out HTTP access was checked on 7 November 2026. Availability on every campus network is not guaranteed. All essential task instructions and feedback are supplied locally.

Exercise answer key

Attempt the original guide exercises first. Match each question number with the explained answer below. Equivalent correct explanations and clearly stated alternative assumptions are acceptable.

Chapter 1 — The Fundamentals

Answer 1.1a1

79. The sign bit is 0, so add the weights 64 + 8 + 4 + 2 + 1.

Answer 1.1a2

−102. In eight-bit two's complement, −128 + 16 + 8 + 2 = −102. Alternatively invert 10011010 and add 1 to obtain 01100110 = 102, then negate.

Answer 1.1a3

11001011. +53 is 00110101; invert to 11001010 and add 1. Check: −128 + 64 + 8 + 2 + 1 = −53.

Answer 1.1a4

−128 through +127 inclusive: −2^7 to 2^7 − 1. There are 256 distinct eight-bit patterns.

Answer 1.1b1

0111 1001 1000. Encode the decimal digits 7, 9 and 8 separately; do not convert the entire value to ordinary binary.

Answer 1.1b2

9776. The four nibbles represent decimal 9, 7, 7 and 6.

Answer 1.1c

A decimal digital display can store each displayed digit as one BCD nibble. Some decimal arithmetic systems also use BCD; it uses more bits than ordinary binary for most multi-digit values.

Answer 1.2a1

Sampling resolution (bit depth) is the number of bits used to encode the amplitude of each audio sample.

Answer 1.2a2

More bits per sample provide more quantisation levels, reducing amplitude quantisation error for a given signal range. This increases raw storage and does not itself increase the number of time samples per second.

Answer 1.2b1

For a stored bitmap, pixel dimensions describe the number of pixels across and down. In display or print contexts, resolution can also mean pixel density. State the intended meaning and units.

Answer 1.2b2

4 bits per pixel, because 2^4 = 16 colour codes.

Answer 1.2b3

256 colours require 8 bits = 1 byte per pixel. 16,384 × 512 = 8,388,608 bytes. Divide by 1,073,741,824 bytes/GiB to obtain 0.0078125 GiB (8 MiB). This excludes palette, header and compression.

Answer 1.3

PC → MAR; memory[MAR] → MDR; increment PC; MDR → CIR; decode; execute. This is the supplied one-word teaching model. Implementations can overlap operations and use different instruction lengths.

Answer 1.4

A backup permits recovery after accidental deletion, corruption, device failure or an attack. Keep a separate recoverable copy with appropriate access controls and test restoration; synchronisation alone can also synchronise a deletion.

Answer 1.5

An assembler translates assembly operations and labels into machine instructions. A compiler translates source into another representation, such as native instructions or bytecode. An interpreter executes operations through a runtime. Real implementations can combine these approaches; a language is not inherently limited to one.

Chapter 2 — Behaviorism & Behaviorist AI

Answer 2.1

Classical conditioning associates a cue with another stimulus so the cue can elicit a response. Operant conditioning changes the likelihood of an action through its consequences. A bell paired with food and a voluntary action followed by a reward illustrate the difference. They are analogies, not proof that numerical RL reproduces animal experience.

Answer 2.2

A thermostat is a fixed reflex example: environment = room; stimulus = measured temperature below the set point; response = switch heating on. With unchanged thresholds it reacts but does not learn a policy from rewards.

Answer 2.3

Subsumption coordinates behaviour modules with defined priority, inhibition or suppression relationships. Obstacle avoidance may suppress a wandering command. Useful behaviour can emerge without a single detailed central map, although practical robots can combine reactive modules with mapping and planning.

Answer 2.4a

The agent chooses an action using its policy; the environment produces the next state and reward according to the task's dynamics. An MDP specifies states, actions, transition probabilities, rewards and discounting. A policy maps states to actions or action probabilities; it is not the reward function.

Answer 2.4b

Exploration obtains information about insufficiently tested actions. Exploitation chooses an action using the best current value estimate, which may include future rewards. Always exploiting can lock in an inferior early estimate; always exploring can waste known good actions. Physical exploration needs safety constraints.

Chapter 3 — Hardware

Answer 3.1a

Primary storage includes working memory and, in some classifications, ROM; it is accessed through the processor's memory system. Main RAM is normally volatile and relatively fast. Secondary storage such as SSD or HDD is non-volatile and usually slower to access. Data normally moves into working memory before ordinary processing. "All primary storage is volatile" is false.

Answer 3.1b

Magnetic: hard disk drive or magnetic tape. Optical: CD, DVD or Blu-ray. Solid-state: SSD or USB flash drive.

Answer 3.2a

SRAM uses a bistable transistor cell and does not need periodic refresh while powered. DRAM stores charge that leaks and needs refresh. Both normally lose data without power. SRAM is fast but less dense and costlier per bit; DRAM supports larger main memories.

Answer 3.2b

Mask ROM is fixed during manufacture. PROM can normally be programmed once. EPROM can be erased with ultraviolet light and reprogrammed; EEPROM and flash instead use electrical erasure. These names refer to different technologies, not one universal write process.

Answer 3.3

Touchscreen = output display plus touch input. SSD = storage, with read/write data transfers. Barcode reader = input. A device can legitimately have more than one role.

Answer 3.4

For a greenhouse, a temperature sensor provides readings to a controller; when the temperature exceeds a defined threshold, the controller drives a fan actuator. Measure again to close the loop. A lower switch-off threshold can prevent rapid on/off chatter. This is fixed-rule control unless a learning method actually updates the decision policy.

Answer 3.5a

RISC families traditionally emphasise relatively regular, simple instruction encodings and load/store operations. CISC families traditionally provide more varied encodings and operations. Modern implementations blur boundaries and may translate complex instructions into micro-operations. Neither label alone proves speed or efficiency.

Answer 3.5b

Pipelining overlaps stages of different instructions to increase throughput. In an ideal four-stage pipeline, three independent instructions need 4 + 3 − 1 = 6 cycles, rather than 12 sequential stage cycles. A data dependency, resource conflict or branch can cause stalls; one instruction still traverses all four stages.

Answer 3.5c

PC holds the next instruction address; ACC holds an intermediate arithmetic result in the teaching model; MDR holds a value or instruction being transferred between memory and the processor. A store address belongs in MAR, not automatically in ACC.

Answer 3.6

A hardware interrupt is a request from hardware for service. When an enabled request is accepted, the system saves the execution state it needs, transfers control to an interrupt service routine, then restores state and resumes. Masking, priorities and architecture affect when this occurs.

Answer 3.7

AND: 00→0, 01→0, 10→0, 11→1. OR: 00→0, 01→1, 10→1, 11→1. NOT: 0→1, 1→0. The usual AND symbol has a flat input side and rounded output side; OR has a curved input side; NOT is a triangle with an inversion circle. Named rectangular gate symbols are also valid if used consistently.

Chapter 4 — Networking

Answer 4.1a

A LAN connects a local site, often under one organisation's control. A WAN connects distant sites across wider geography and may depend on carriers. WAN paths often introduce longer propagation paths, but equipment, traffic and access links determine actual throughput and delay; scope alone does not guarantee a speed.

Answer 4.1b

Client–server centralises a service, making policy and backup management easier but creating capacity and availability responsibilities. Peers can both request and provide resources, spreading work but complicating discovery, trust and consistency. Both can scale with suitable designs; neither is automatically unlimited.

Answer 4.1c

A thick client performs substantial local processing and may keep local data. A thin client depends more on a remote server for computation and storage. Thin clients simplify some central management but depend more on network availability; actual systems can be hybrids.

Answer 4.1d

A public cloud uses provider-operated shared infrastructure, often lowering up-front costs but adding provider dependence and shared security responsibilities. A private cloud is dedicated to one organisation and offers more direct control, but the organisation must fund and operate or contract capacity and security. Neither is inherently secure merely because of its label.

Answer 4.2

Legacy shared, half-duplex Ethernet uses CSMA/CD to detect a collision and retry after backoff. Wi-Fi uses collision avoidance, including channel sensing, waiting and acknowledgements. Modern switched full-duplex Ethernet links normally avoid shared-medium collisions; congestion and queues can still occur.

Answer 4.3a

In https://example.org/guide/chapter1, https is the scheme, example.org is the host name and /guide/chapter1 is the path. A URL can also include a port, query and fragment. It identifies how and where to request a resource, not necessarily a fixed physical machine.

Answer 4.3b

DNS resolves names through a distributed hierarchy and caches. A resolver may return an address record for the requested host; the application can then use that address to contact a server. DNS does not itself download the web page, and cached records have limited lifetimes.

Answer 4.4a

Application: services and application messages. Transport: communication between application endpoints, such as TCP streams or UDP datagrams. Internet: IP addressing and routing across networks. Link/network-access: delivery over a local link, such as Ethernet or Wi-Fi. Layer names vary slightly between teaching models.

Answer 4.4b

HTTP requests and transfers web resources. FTP transfers files but its traditional form lacks confidentiality. POP3 retrieves email, often for local storage. IMAP manages and synchronises email kept on a server. Secure variants or TLS are required where credentials and content need protection.

Answer 4.5a

Circuit switching reserves communication resources along a path for a session. Capacity can remain allocated even when no useful data is being sent, giving a different sharing trade-off from statistical packet multiplexing.

Answer 4.5b

Under the stated reserved-capacity requirement, a circuit can provide predictable allocated capacity without competing for that same reservation during the call. It may waste idle capacity and needs setup. Modern Internet video calls normally use packet networks; the exercise is a hypothetical trade-off, not a claim about usual Internet practice.

Answer 4.5c

Benefits: bursty users can share link capacity efficiently, and packets can be forwarded along available paths. Drawback: queueing, loss or reordering can create variable delay and require handling. No route change or reliability is guaranteed merely by using packets.

Answer 4.6a

Security protects systems and data against unauthorised access or harm. Privacy concerns appropriate collection and use of personal information. Integrity concerns accuracy and resistance to unauthorised alteration. Encrypted personal data can still be collected without a legitimate purpose, so confidentiality alone does not settle privacy.

Answer 4.6b

Validation checks whether data satisfies rules such as format or range; a plausible wrong date can pass. Verification checks agreement with an original or expected copy, such as double entry or a transfer checksum. A checksum detects some changes but does not establish authenticity against an active attacker.

Answer 4.6c

Examples: least-privilege access controls, timely security updates, strong authentication, firewalls, encryption, protected backups and malware detection. Explain the threat addressed; no single mechanism stops every threat.

Answer 4.7a

Symmetric cryptography uses a shared secret and is efficient for bulk data, but securely distributing and protecting that secret matters. Asymmetric mechanisms use a public/private key pair for particular tasks such as key establishment or signatures. Both require correct algorithms and key handling; a public key is not itself secret.

Answer 4.7b

The sender signs with a private signing key; recipients verify with the matching public key. A valid signature supports origin and integrity relative to trust in that key, but does not hide the message. Encrypting for confidentiality is a distinct operation. A copied valid old signed message may still require replay protection.

Answer 4.7c

SSL is the obsolete predecessor to TLS. TLS is used today for protected transport in HTTPS and other application protocols, providing confidentiality and integrity with authentication according to configuration. TLS does not make an endpoint safe or prevent a legitimate user from misusing received data.

Chapter 5 — Applications

Answer 5.1

A smart system connects sensing, computation, communication and action to a goal. For irrigation, a moisture sensor supplies readings, software compares them with the policy, a network reports status and a valve changes water flow. It need not use machine learning simply because it is connected.

Answer 5.2a

Real-time processing means meeting a defined timing deadline, not merely being fast on average. Communication, queueing, computation and actuator delays all contribute. At 30 m/s, an extra 0.2 s delay means 6 m travelled before a response; this is delay distance, not total braking distance.

Answer 5.2b

Automation reduces direct manual operation of a task. It can improve consistency and throughput, but can also repeat a wrong rule at scale, fail on unusual inputs or change workers' monitoring responsibilities. Assess the actual process rather than assuming all errors disappear.

Answer 5.3a

An RL algorithm uses transitions and rewards to update policy parameters or value estimates, aiming at expected cumulative reward. A deployed agent may use a fixed learned policy without updating it on every action. Correctly associate feedback with the state and action that produced it.

Answer 5.3b

An actuator turns an electrical control signal into physical action: a motor moves a wheel, a valve changes flow, or a heater produces heat. The controller must respect limits and use feedback where needed; a software command is not proof the requested movement occurred.

Answer 5.4a

Human-AI interaction concerns how people work with AI-driven systems. Uncertainty, opaque decisions, changing performance and delegated actions add challenges to general interface design. Interfaces should communicate limits, support correction and provide meaningful human control.

Answer 5.4b

A user may over-trust an unexplained "safe" decision and ignore a hazard, or be unable to recover when an autonomous action is wrong. Display relevant status and uncertainty, explain the available action and offer an effective override or recovery process.

Answer 5.5a

Ethical AI considers consequences for people as well as performance: fairness across groups, explanations appropriate to users, privacy and accountable responsibility. Trade-offs must be evaluated in context, with monitoring and redress. A fairness score alone is not a complete ethical judgement.

Answer 5.5b

Opportunity: remote monitoring can find faults and reduce wasted energy. Risk: compromised devices can expose personal data or accept dangerous commands. Minimise collected data, authenticate access and maintain update and incident-response processes.

Chapter 6 — Strengths & Limitations

Answer 6.1

An embedded device has limited memory for weights and working data, compute time for deadlines, power from its supply and thermal capacity to remove heat. A model that fits storage can still exceed RAM or latency budgets. Consider quantisation or a smaller model, then measure accuracy and worst-case timing on the target device.

Answer 6.2

Scalability means accommodating growth while meeting acceptable service targets. Interoperability means exchanging and correctly using information between systems. A network that can carry many packets still fails interoperability if one sensor sends centimetres and the receiver assumes metres. Test load, schemas, units, versions and identities.

Answer 6.3

Adaptive intelligence changes behaviour appropriately using new evidence or experience. It can respond to changed conditions, but noisy feedback, distribution shifts or unsafe exploration can make adaptation harmful. Test changes against objectives and retain monitoring and recovery controls.

Answer 6.4

Predictive analytics estimates future or unknown outcomes from evidence, such as a likely equipment failure. Behavioural modelling represents actions and interactions, such as how drivers respond to signs. A model can be useful without proving causes; both require relevant data and evaluation on new cases.

Answer 6.5

Resource optimisation selects where and how to compute while balancing latency, accuracy, energy, bandwidth, privacy and cost. Local emergency control can avoid a remote round trip; cloud processing can support large-scale training or reporting. Edge processing still has hardware limits and requires maintenance.

Answer 6.6

Explainability makes relevant aspects of model behaviour understandable to a particular audience. Deep RL can be difficult to explain because many learned parameters and sequential consequences interact. An approximate post-hoc explanation may be useful, but is not a guarantee of the model's true causal reasoning.

Answer 6.7

Security concerns include unauthorised commands and replay or manipulation of sensor messages. Ethical concerns include unfair impacts, privacy loss and unclear responsibility for harm. Combine authentication, freshness checks, monitoring, human oversight and explicit accountability. Valid alternative risks and controls should earn credit when well justified.

ROVERLAB — Build, Connect, Train

Interactive companion: Play ROVERLAB

Provider: Blue Impulse. **Time:** approximately 5–10 minutes per experiment. **Preparation:** read Chapter 2 — Behaviorism & Behaviorist AI, Chapter 3 — Hardware, and Chapter 4 — Networking; use Chapter 5 — Applications to connect the systems. No programming experience or account is required.

Your challenge

Build an exploration rover that can sense obstacles, visit a research site, send its findings to base and return. Predict how one changed component or response will affect the mission, then compare the prediction with the event log and results.

1. Choose Exploration rover. In Hardware, inspect the processor, memory, motors, wheels or tracks, sensors and power supply. You may try the starter build, then inspect what each part contributes. Remove the distance sensor once: read why sensing or mission readiness changes, then restore it.

2. Add a radio module and connect the base station. Run the network test. Explain why a connected base station is a different role from the robot's processor.

3. Train at least three rules: obstacle detected → stop, then turn; destination reached → return to base; low battery → return to base. Test a stimulus and give feedback. Compare competing associations by strength and order. The research sensor collects its reading at the site and queues it for transmission through your actual network route; the destination rule tells the drive system to return.

4. Select Survey & return. Resolve the readiness findings, then launch. Pause or step through an obstacle event. Identify the sensor observation, selected rule, actuator action and later packet transmission.

5. Record the result and four scores. Return to training and replace obstacle → stop, then turn with obstacle → move / run. Predict the outcome before launching again. Compare the actual event evidence rather than only the total score.

Observe, explain and extend

Record two rows: configuration, prediction, observed event, result, and explanation. Which component detected the obstacle? Which rule selected the response? What made the wheels move? Which route delivered the report? Why can a compatible rule still be unhelpful?

Expected reasoning: an appropriate sensor makes the obstacle observable; the processor selects a supported rule; the drive system carries out the command. Continuing toward the obstacle can cause a collision. A module alone does not establish a usable route, and a route that works at the start may fail as the robot moves. Feedback changes which competing association wins; it does not make an unsafe command safe or supply missing hardware.

For another experiment, use Search & assist to combine person sensing and a control computer, or Monitor & protect to detect high temperature, stop the process and send a server alert. Chapter 6 — Strengths & Limitations helps explain why a successful toy mission is insufficient evidence for a real autonomous deployment.

Access and offline alternative

Open the link in a modern desktop or mobile browser. After a complete first visit, the local game cache supports offline reloads where service workers are available. Progress is saved in this browser; private browsing or blocked storage can limit persistence. The repository also provides a source ZIP that can be extracted and opened at public/index.html without internet:
https://github.com/user-github-me/cs-ai-project-01/tree/main/apps/roverlab

If the game cannot load, draw this trace on paper: obstacle → distance sensor → processor selects rule → motor response → research reading → radio link → base. Compare stop-and-turn with continuing forward; remove the sensor or radio and mark the exact missing link. This supplies the same core reasoning task without the website.

ROVERLAB uses explicit player-authored stimulus–response associations and simplified feedback. It is not a neural network or a full reinforcement-learning trainer. Range, power and environmental values are fictional teaching values. The source and automated checks are public; a student or teacher playtest remains separate evidence.

Sources, access and revision notes

The six team-supplied chapter PDFs defined the scope. Their notes credit Cambridge International AS & A Level Computer Science but do not identify a full edition or author bibliography; this remains a provenance limitation and no edition has been invented. The new activities, calculations and checkpoints are original AI-assisted teaching examples. The guide's original artwork is user supplied and retains its own third-party rights.

The resources below were inspected as optional extensions and reference support. No external link replaces an explanation, input, question or answer in this snapshot. No student-created website or external platform is required.

R1 — Reinforcement Learning: An Introduction, second edition

Richard S. Sutton and Andrew G. Barto, MIT Press, 2018.

http://incompleteideas.net/book/the-book-2nd.html

Relevant section: Chapter 3 (finite Markov decision processes) and Section 6.5 (Q-learning)

Why use it: Primary textbook support for the agent/environment distinction, expected return and the Q-learning update.

Learner task: After ACT-02, explain in your own words what the bootstrapped target includes, then compare it with the terminal-state case.

Limits: The book is mathematically deeper than this beginner guide. Start with the supplied worked calculation; do not treat the browser page as required activity content.

Offline alternative: The formula, both complete calculations and interpretation are included in ACT-02 and the chapter, so the task works offline.

Access: Optional public web resource; no account required to read the cited page. Signed-out HTTP access was checked on 7 November 2026. Availability on every campus network is not guaranteed. All essential task instructions and feedback are supplied locally.

R2 — Nand to Tetris — Project 5: Computer Architecture

Noam Nisan and Shimon Schocken / Nand to Tetris.

https://www.nand2tetris.org/project05

Relevant section: Project overview, proposed computer architecture and hardware simulator guidance

Why use it: A concrete educational computer architecture connects logic, CPU state and memory.

Learner task: After ACT-01, compare the purpose of an instruction address and a data-memory address with the project's computer model. Name one part the models share and one detail that differs.

Limits: The Hack computer is a different ISA; its instructions and registers must not be substituted directly into this guide's toy program. Software installation is optional and not required for the paper task.

Offline alternative: ACT-01 includes its full program, trace and expected output. ACT-03 provides a complete paper pipeline grid specification.

Access: Optional public web resource; no account required to read the cited page. Signed-out HTTP access was checked on 7 November 2026. Availability on every campus network is not guaranteed. All essential task instructions and feedback are supplied locally.

R3 — RFC 1122 — Requirements for Internet Hosts: Communication Layers

IETF / R. Braden, editor, 1989.

https://www.rfc-editor.org/rfc/rfc1122

Relevant section: Section 1.1.3 (Internet protocol suite) and the link, IP and transport sections

Why use it: An authoritative foundation for the layered Internet model used in Chapter 4.

Learner task: Draw the four teaching layers and place HTTP, TCP, IP and Ethernet/Wi-Fi by role. Explain why a large link capacity does not set an end-to-end response time.

Limits: This is a technical historical standards document; current protocols have later updates. Use it for layer roles, not as a complete modern security checklist.

Offline alternative: The chapter and ACT-04 contain the layer explanations and all delay inputs/results offline.

Access: Optional public web resource; no account required to read the cited page. Signed-out HTTP access was checked on 7 November 2026. Availability on every campus network is not guaranteed. All essential task instructions and feedback are supplied locally.

R4 — RFC 8446 — The Transport Layer Security (TLS) Protocol Version 1.3

IETF / Eric Rescorla, 2018.

https://www.rfc-editor.org/rfc/rfc8446

Relevant section: Section 1 (Introduction) and Section 8 (0-RTT and anti-replay)

Why use it: Distinguishes transport confidentiality/integrity from application-level decisions, including replay considerations.

Learner task: After ACT-05, explain why authenticated transport alone does not decide whether an old command is still appropriate to execute.

Limits: The Python receiver is a validation demonstration, not a TLS implementation. Do not implement cryptography from this worksheet.

Offline alternative: ACT-05 includes the packet fields, state transitions and rejection explanations; no network service or account is needed.

Access: Optional public web resource; no account required to read the cited page. Signed-out HTTP access was checked on 7 November 2026. Availability on every campus network is not guaranteed. All essential task instructions and feedback are supplied locally.

R5 — Artificial Intelligence Risk Management Framework (AI RMF 1.0)

NIST, NIST AI 100-1, January 2023.

https://www.nist.gov/itl/ai-risk-management-framework

Relevant section: Framework functions Govern, Map, Measure and Manage; trustworthy AI characteristics

Why use it: Connects measured performance with context, impacts, monitoring and responsibility.

Learner task: For ACT-06, assign one concrete action to each framework function: identify an owner, describe the operating domain, measure a relevant failure and plan a response.

Limits: A voluntary risk-management framework is not certification that a product is safe, fair or legally compliant. Context and evidence still matter.

Offline alternative: ACT-06 supplies the design table and expected trade-off reasoning. A suitable four-action response is: Govern — name an accountable operator; Map — define the campus operating domain; Measure — test missed deadlines and uneven accuracy; Manage — maintain fallback and an incident response process.

Access: Optional public web resource; no account required to read the cited page. Signed-out HTTP access was checked on 7 November 2026. Availability on every campus network is not guaranteed. All essential task instructions and feedback are supplied locally.

R6 — The Unicode Standard — technical introduction

The Unicode Consortium.

https://www.unicode.org/standard/principles.html

Relevant section: Unicode design principles and character encoding model

Why use it: Supports the difference between a code point and a variable-length encoding such as UTF-8.

Learner task: Explain why one character is not necessarily one byte, and why BCD digits and text digits need an agreed representation.

Limits: Full text processing includes combining characters and grapheme clusters beyond this chapter. A displayed symbol is not always one code point.

Offline alternative: Chapter 1 defines character sets, code points and encodings and includes the BCD/ordinary-binary comparison.

Access: Optional public web resource; no account required to read the cited page. Signed-out HTTP access was checked on 7 November 2026. Availability on every campus network is not guaranteed. All essential task instructions and feedback are supplied locally.

Responsible AI-use record

Neogpt/Codex assisted on 7 November 2026 with reconciling the guide into Markdown, developing activities and answers, drafting the Test Bank, generating PDFs/DOCX and verifying package structure. The user asked to follow rules/Stage2_All_Instructions.md. Calculations and supplied code outputs were checked locally; source/resource access and document checks are recorded in the submission evidence. These checks are not independent human peer review. Significant interaction excerpts belong in the project and personal logs.